

重庆大学计算机专业核心课程《机器学习》

第8章 集成学习

授课教师：郑林江

第8章 集成学习

➡ 8.1 个体与集成

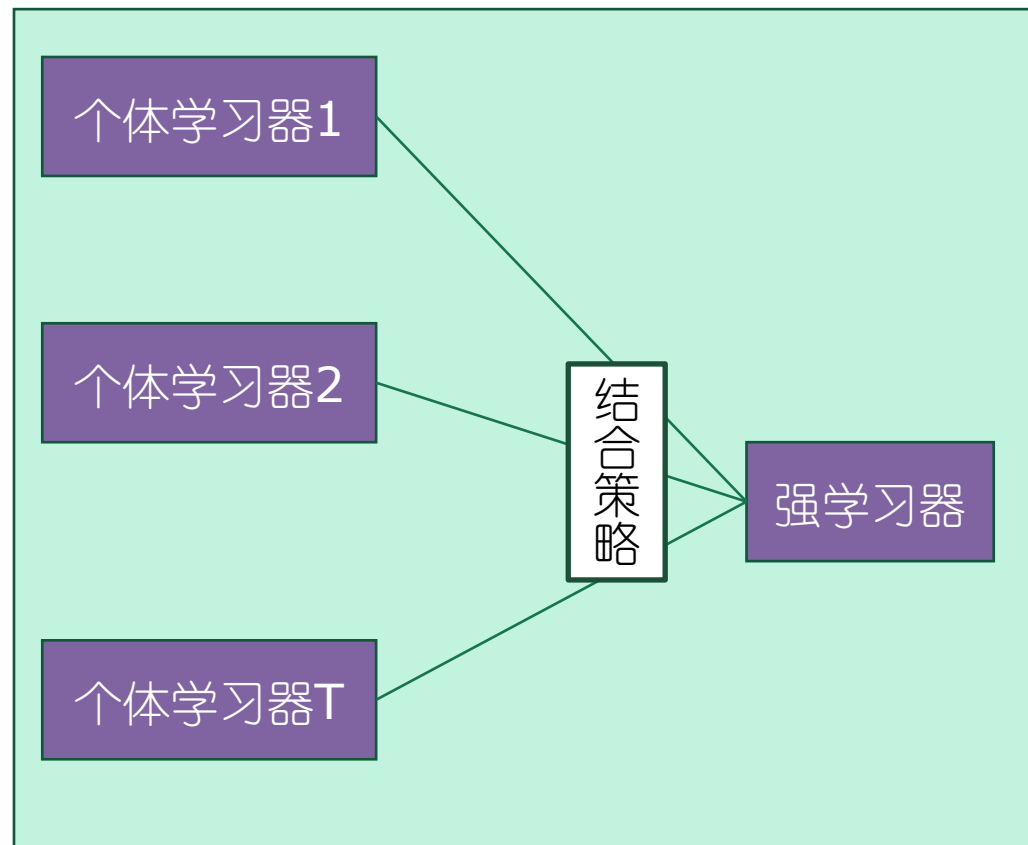
8.2 Boosting

8.3 Bagging与随机森林

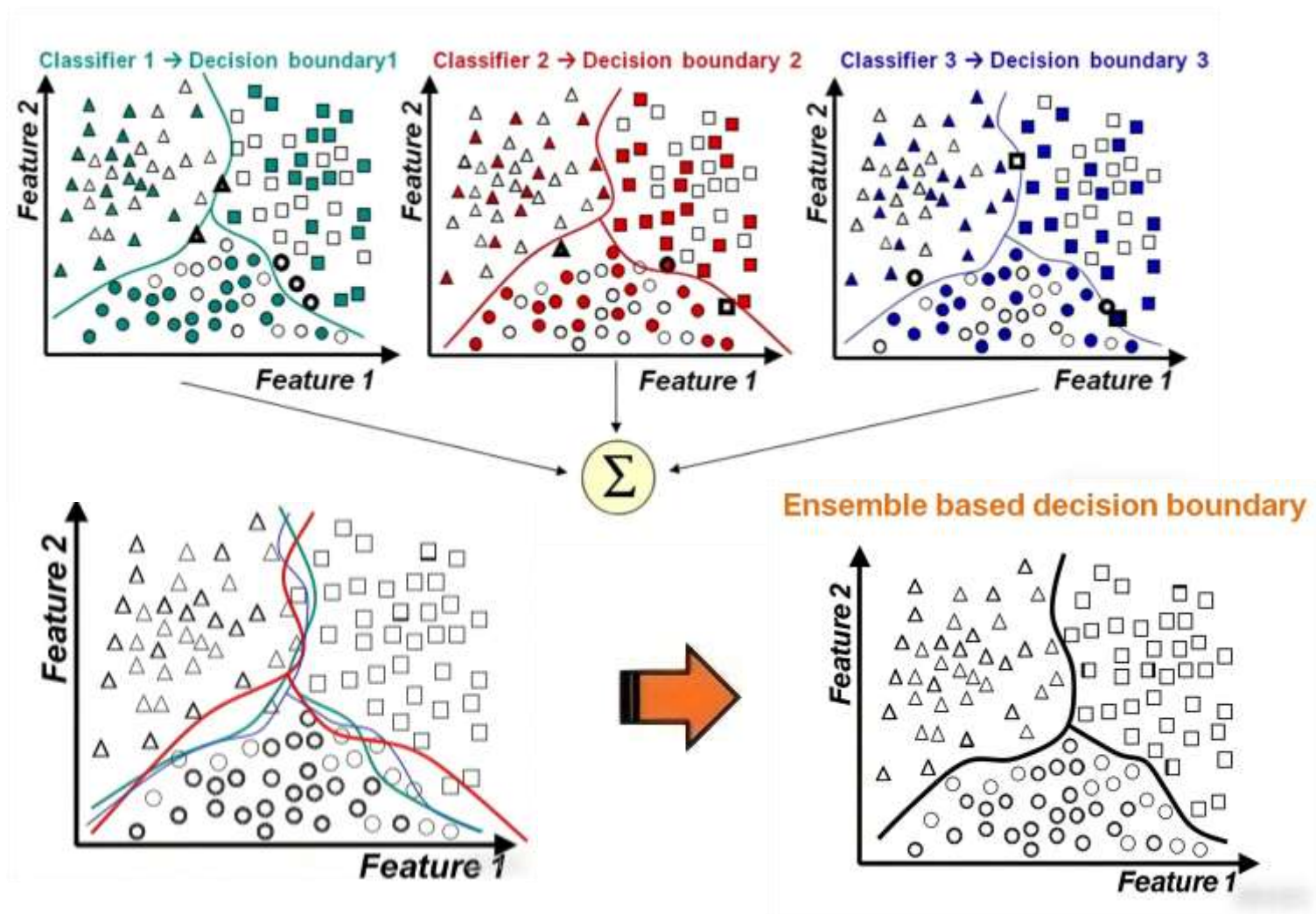
8.4 结合策略

8.5 多样性

集成学习



集成学习



集成学习

□ 集成学习：一种组合基础模型的技术框架

- 通过整合多个单一模型的预测结果，实现更优的整体效果，相比单一模型，它能借助“集思广益”的优势，有效降低过拟合风险。

□ 集成学习两大关键问题：

- 一是如何构建出多个个体学习器；
- 二是如何选定合适的结合策略，把这些个体学习器整合为性能更强的强学习器。

集成学习

□ 如何得到若干个个体学习器：

- 同质个体学习器：所有基础模型属于同一类，比如全部用决策树、全部用神经网络，模型种类单一。
- 异质个体学习器：基础模型为不同种类，比如搭配支持向量机、逻辑回归、朴素贝叶斯等多种模型，再通过结合策略整合输出。

同质个体学习器

同质个体学习器是目前应用最广泛的类型，日常提及的集成学习方法大多指代这类，其中使用最普遍的基础模型是CART决策树与神经网络。

□ 这类学习器还可根据个体间依赖关系细分为两类：

- 强依赖关系（串行生成）：个体学习器需按顺序依次生成，前后关联紧密，代表算法为boosting系列算法。
- 无强依赖关系（并行生成）：个体学习器可同时独立生成，互不干扰，代表算法为bagging系列算法。

基本原理

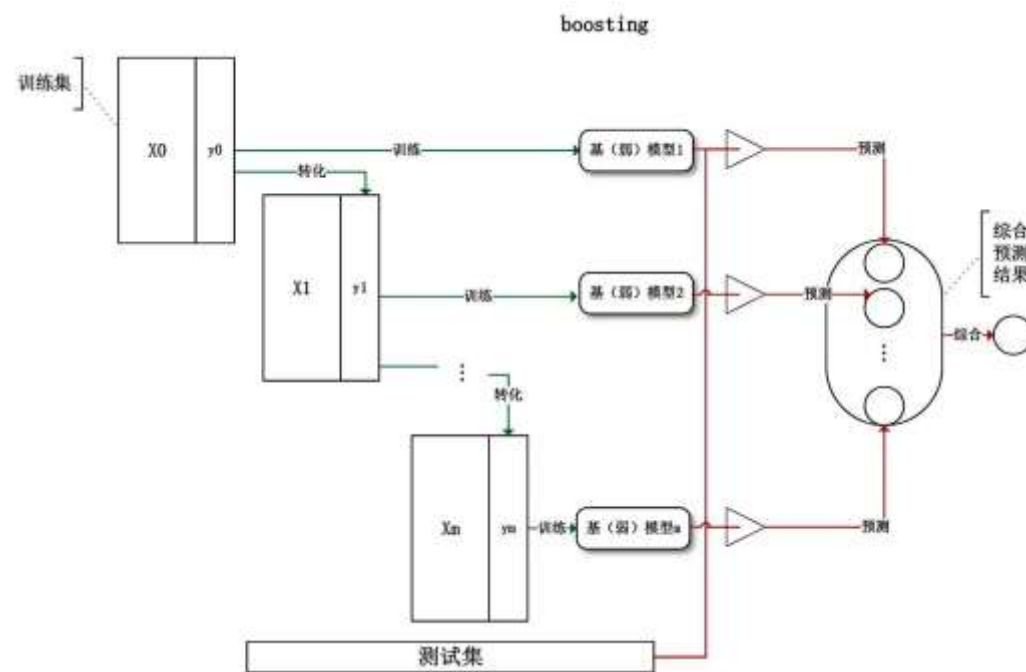
- 无强依赖关系（并行生成）：个体学习器可同时独立生成，互不干扰，代表算法为bagging系列算法



代表性算法：RandomForest

集成学习

- 强依赖关系（串行生成）：个体学习器需按顺序依次生成，前后关联紧密，
 - 代表算法为boosting系列算法——训练过程为阶梯状，基模型按次序一一进行训练（实现上可以做到并行），基模型的训练集按照某种策略每次都进行一定的转化。对所有基模型预测的结果进行线性综合产生最终的预测结果

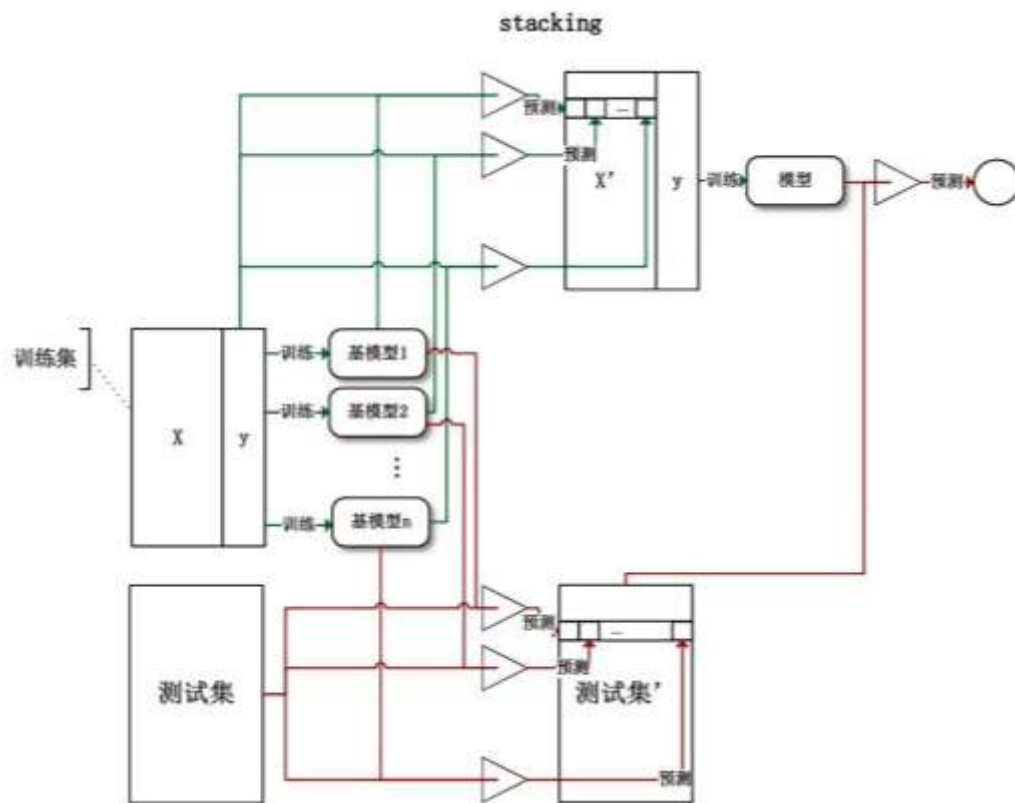


代表性算法：Adaboost、GBDT、XGBOOST

集成学习

- **Stacking**: 将训练好的所有基模型对训练集进行预测，第 j 个基模型对第 i 个训练样本的预测值将作为新的训练集中第 i 个样本的第 j 个特征值，最后基于新的训练集进行训练。同理，预测的过程也要先经过所有基模型的预测形成新的测试集，最后再对测试集进行预测。

- 由于使用了两层的学习模型，**stacking**理论上可以让算法预测更准确，但是需要花更多的时间来调参。因为第一层的模型输出对第二层的模型参数影响很大。



投票分类器(Voting Classifiers)

- 对于一个训练集，有很多分类器，比如说Logistic、KNN、SVM等。对于一个样本，给出所有分类器的分类结果，然后利用这个结果对样本的分类进行预测
 - **hard voting classifier**：不考虑分类器的差别，比如说准确性等，直接取投票数最多的类别，将其作为最后的对于该样本的分类结果
 - **soft voting classifier**：利用所有分类器给出的各个类别的概率，最后利用各个类别的概率之和进行预测，soft voting的准确率要略微高于hard voting，因为它考虑了每个模型的不同。

投票分类器(Voting Classifiers)

- 在很多情况下，投票分类器的精度会比集合里最好的分类器的精度还要高(对于大多数测试集)，因为这种集成方法提高了模型鲁棒性。
- 当集成方法中的基学习器之间互相独立时，集成方法的效果会更好。
(不同的方法，得出了了相似的结论)

Bagging和Boosting

- 训练多基学习器：可以选择很多不同的训练算法，也可以对于每个基学习器，都选择相同的训练算法，但是对于每个基学习器，都用不同的子数据集或者子特征集进行训练
 - 有放回抽样 **Bagging**：即数据集允许重复（对于每一个基学习器）
 - 无放回抽样**Pasting**：即数据不允许重复
- 得到很多基学习器之后，就可利用统计方法进行分类或者回归
- **bagging**与**randomforest**是集成学习中的两个比较出名的算法

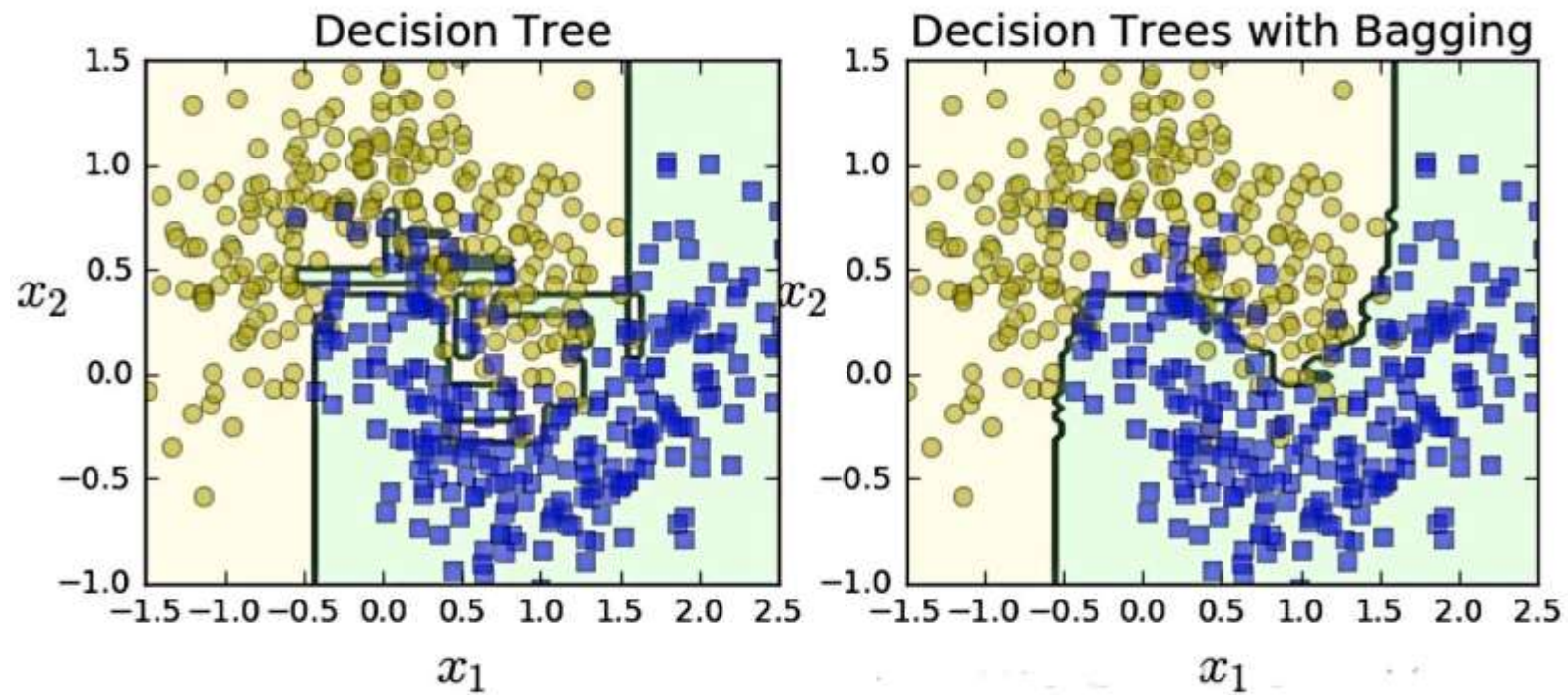
Bagging和Boosting

□ sklearn中的BaggingClassifier可以实现Bagging

- n_estimators: 基学习器的数量
 - max_samples: 每个基学习器中的样本数，如果是整形，则就是样本个数；如果是float，则是样本个数占所有训练集样本个数的比例
 - bootstrap : 是否采用有放回抽样(bagging)，为True表示采用，否则为pasting。默认为True
 - n_jobs: 并行运行的作业数量。-1时，个数为处理器核的个数
 - oob_score: 为True时，对模型进行out-of-bag的验证，即在一个基学习器中，没有用于训练的数据用于验证

□ 相对于单个的决策树，Bagging方法得到的分类边界更加平滑

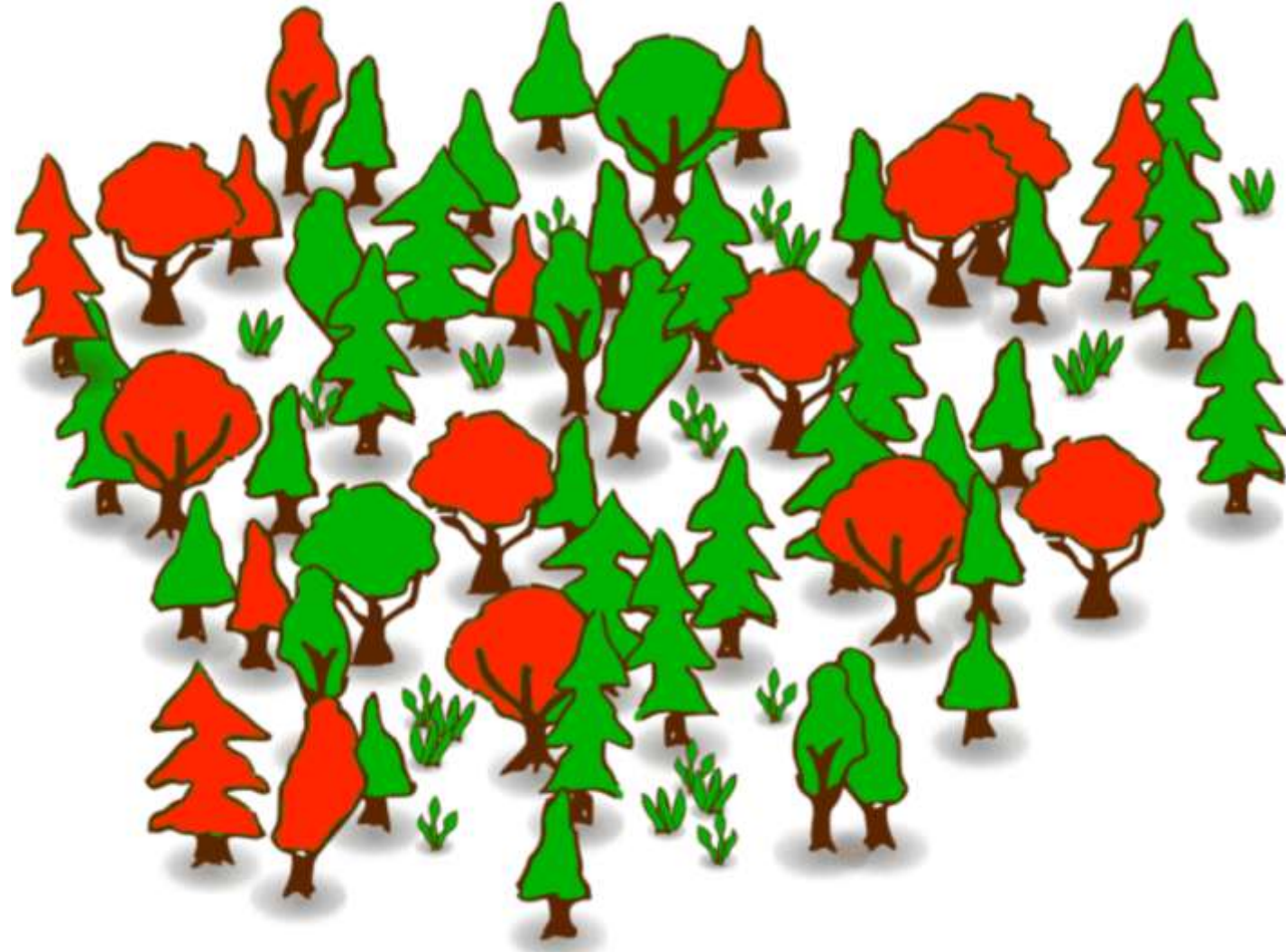
Bagging和Boosting



随机森林

- ❑ 集成学习是将多个模型进行组合来解决单一的预测问题。它的原理是生成多个分类器模型，各自独立地学习并作出预测。这些预测最后结合起来得到预测结果，因此和单独分类器的结果相比，结果一样或更好。
- ❑ 随机森林是集成学习的一个分支，因为它依靠于决策树(CART)的集成。
- ❑ 随机森林是将其他的模型进行聚合， 从其名称也可以看出，随机森林聚合的是分类（或回归）树。一颗决策树是由一系列的决策组合而成的，可用于数据集的观测值进行分类 。

随机森林

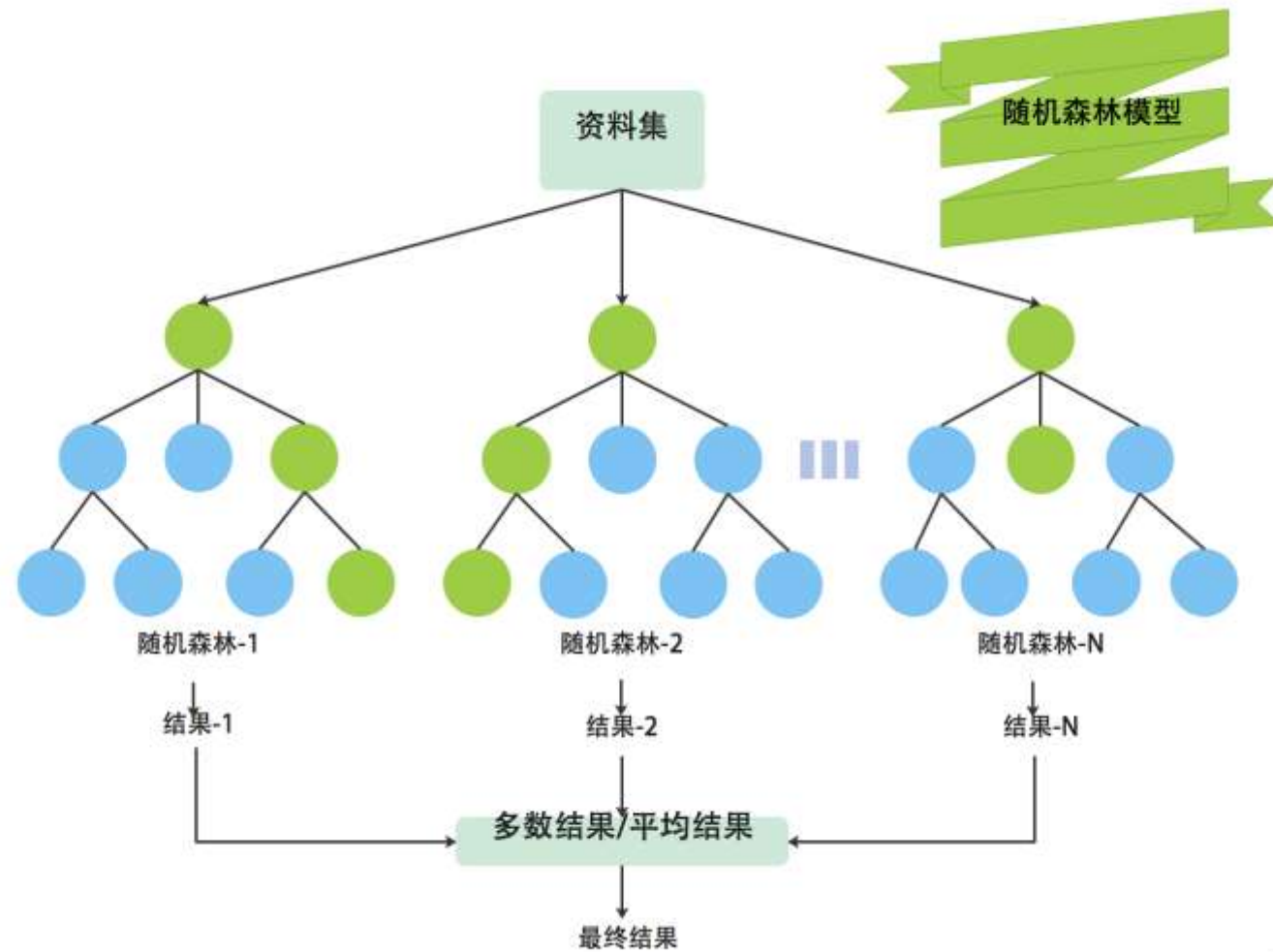


随机森林

□ 随机森林的生成方法：

- 从样本集中通过重采样的方式产生 n 个样本
- 假设样本特征数目为 a ，对 n 个样本选择 a 中的 k 个特征，用建立决策树的方式获得最佳分割点
- 重复 m 次，产生 m 棵决策树
- 多数投票机制来进行预测

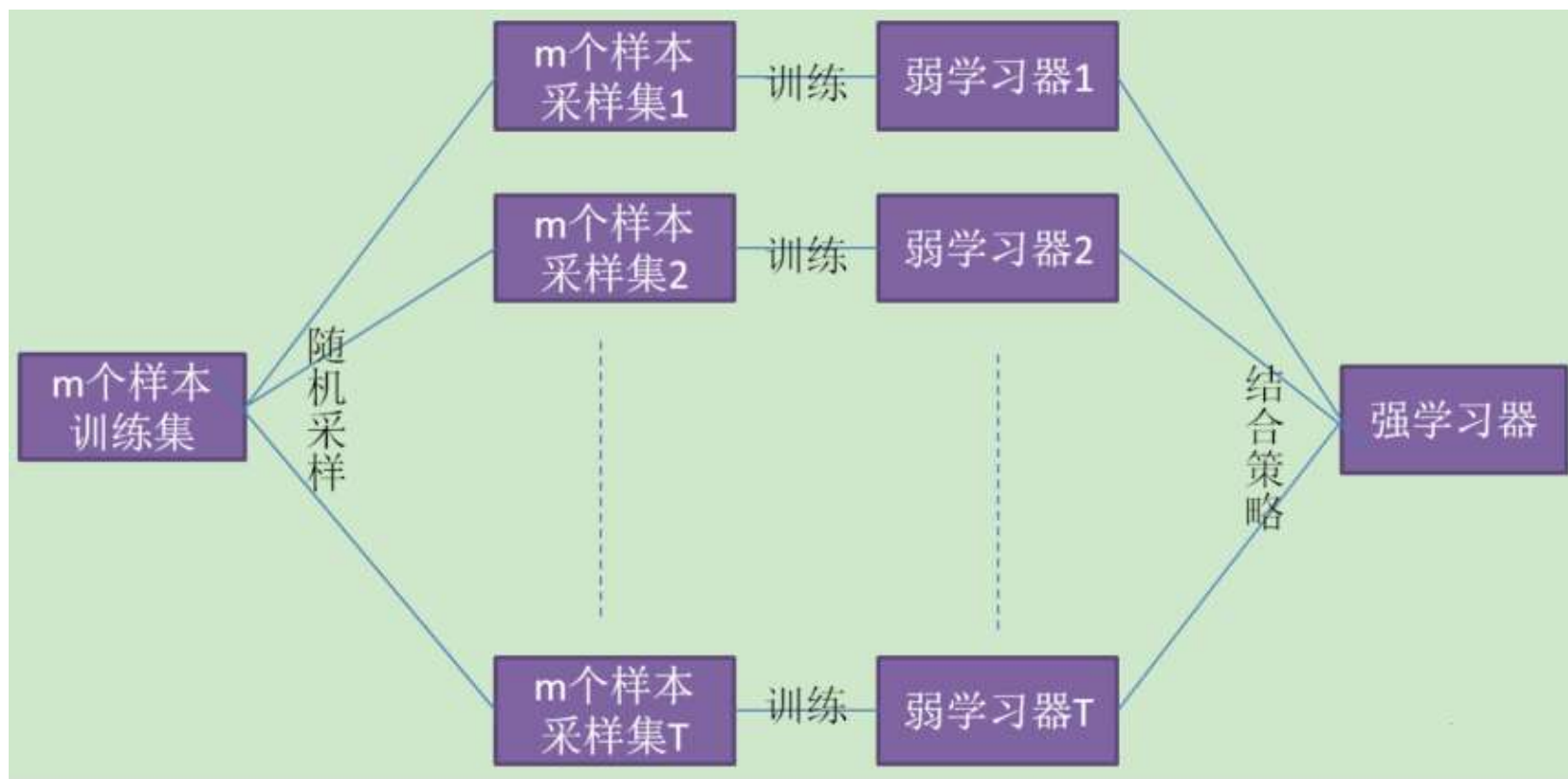
随机森林



随机森林

- 随机森林算法将自动创建随机决策树群。
 - 由于这些树是随机生成的，大部分的树(甚至 99.9%)对解决分类或回归问题是没有意义。但是少数非常好的决策树也随之一起生成了。
- 预测阶段：
 - 新的观察值随着决策树自上而下走下来并被赋予一个预测值或标签。一旦森林中的每棵树都有了预测值或标签，所有的预测结果将被归总到一起，所有树的投票返回, 做为最终的预测结果。
- 简单来说，99.9%不相关的树做出的预测结果涵盖所有的情况，这些预测结果将会彼此抵消。少数优秀的树的预测结果将会脱颖而出，从而得到一个好的预测结果。

Bagging



Bagging

- ❑ Bagging的特点在Bootstrap，子采样是放回采样。
- ❑ bagging对于弱学习器没有限制，最常用的是决策树和神经网络。
- ❑ bagging的集合策略也比较简单：
 - 对于分类问题，通常使用简单投票法，得到最多票数的类别或者类别之一为最终的模型输出。
 - 对于回归问题，通常使用简单平均法，对T个弱学习器得到的回归结果进行算术平均得到最终的模型输出。
- ❑ 由于Bagging算法每次都进行采样来训练模型，因此泛化能力很强，对于降低模型的方差很有作用。当然对于训练集的拟合程度就会差一些，也就是模型的偏倚会大一些。

随机森林

- 随机森林是一种特殊的bagging方法，它将决策树用作bagging中的模型。
 - 首先，用bootstrap方法生成 m 个训练集
 - 然后，对于每个训练集，构造一颗决策树，在节点找特征进行分裂时，并不对所有特征找到能使得指标（如信息增益）最大的，而是在特征中随机抽取一部分特征，在抽到的特征中间找到最优解，应用于节点，进行分裂。
- 随机森林的方法由于有了bagging，也就是集成的思想在，实际上相当于对于样本和特征都进行了采样（如果把训练数据看成矩阵，就像实际中常见的那样，那么就是一个行和列都进行采样的过程），所以可以避免过拟合。prediction阶段的方法就是bagging的策略，分类投票，回归均值。

随机森林

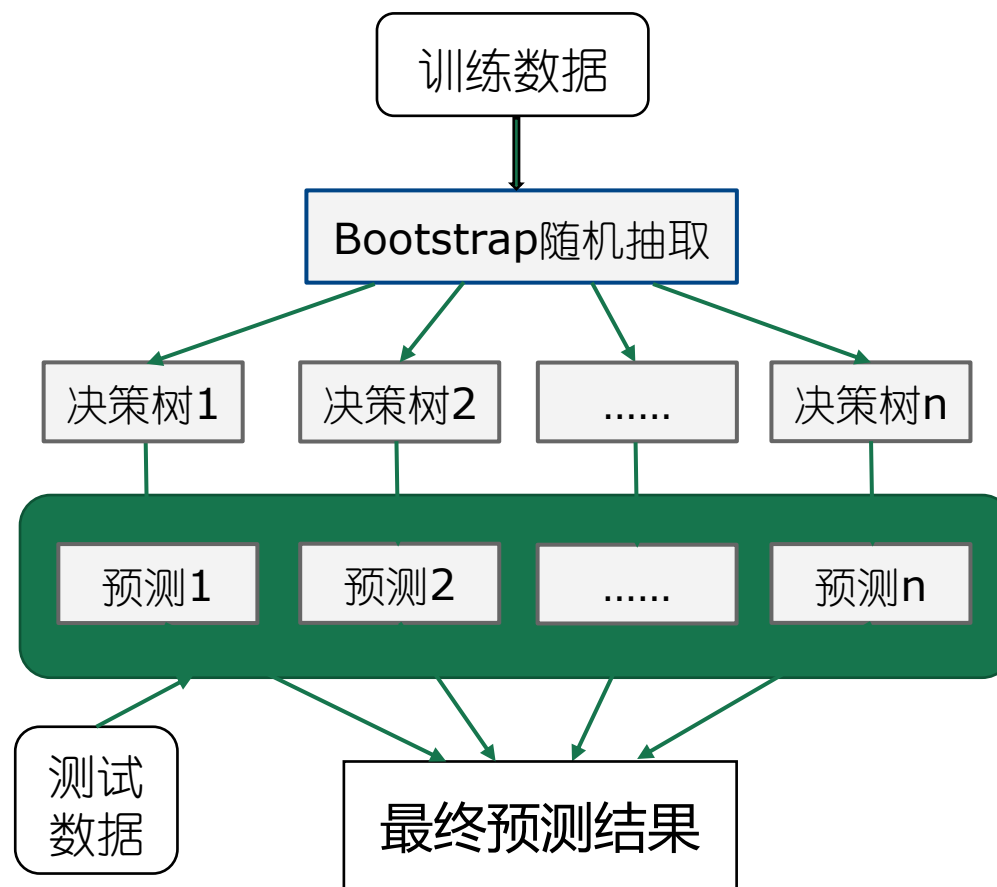
- 随机森林是Bagging算法的进化版，思想仍然是bagging，但是进行了独有的改进：
 - 首先，RF使用了CART决策树作为弱学习器，
 - 第二，在使用决策树的基础上，RF对决策树的建立做了改进，对于普通的决策树，会在节点上所有的 n 个样本特征中选择一个最优的特征来做决策树的左右子树划分，但是RF通过随机选择节点上的一部分样本特征，这个数字小于 n ，假设为 n_{sub} ，然后在这些随机选择的 n_{sub} 个样本特征中，选择一个最优的特征来做决策树的左右子树划分。这样进一步增强了模型的泛化能力。

随机森林

- 随机森林是Bagging算法的进化版，思想仍然是bagging, 但是进行了独有的改进：

随机森林可概括为四个部分：

1. 随机选择样本（放回抽样）
2. 随机选择特征
3. 构建决策树
4. 随机森林投票（平均）



随机森林

□ RF的主要优点有：

- 训练可以高度并行化，对于大数据时代的大样本训练速度有优势。
- 由于可以随机选择决策树节点划分特征，这样在样本特征维度很高的时候，仍然能高效的训练模型。
- 在训练后，可以给出各个特征对于输出的重要性
- 由于采用了随机采样，训练出的模型的方差小，泛化能力强。
- 相对于Boosting系列的Adaboost和GBDT， RF实现比较简单。
- 对部分特征缺失不敏感。

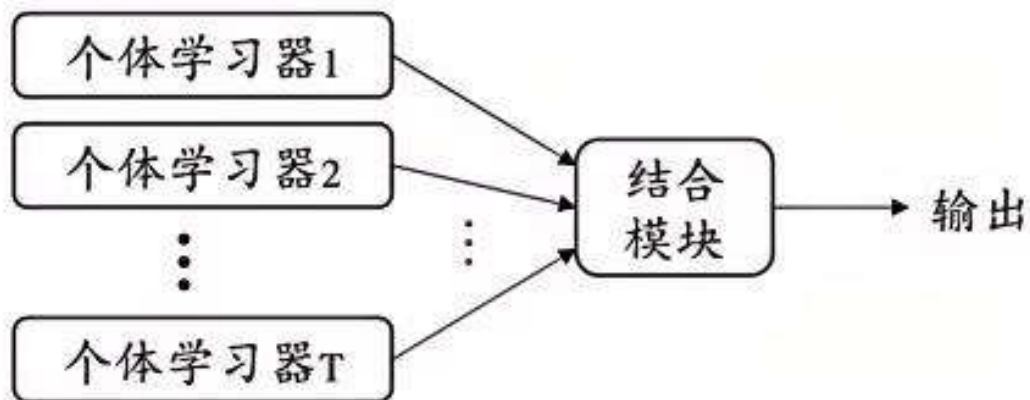
□ RF的主要缺点有：

- 在某些噪音比较大的样本集上， RF模型容易陷入过拟合。
- 取值划分比较多的特征容易对RF决策产生更大影响，从而影响模型拟合效果。

个体与集成

集成学习(ensemble learning):

通过构建并结合多个学习器来提升性能



例子

- 考虑一个简单的例子，在二分类问题中，假定3个分类器在三个样本中的表现如下图所示，其中✓表示分类正确，X号表示分类错误，集成的结果通过投票产生。

测试例1 测试例2 测试例3				测试例1 测试例2 测试例3				测试例1 测试例2 测试例3			
h_1	✓	✓	×	h_1	✓	✓	×	h_1	✓	×	×
h_2	×	✓	✓	h_2	✓	✓	×	h_2	×	✓	×
h_3	✓	×	✓	h_3	✓	✓	×	h_3	×	×	✓
集群	✓	✓	✓	集群	✓	✓	×	集群	×	×	×
(a) 集群提升性能				(b) 集群不起作用				(c) 集群起负作用			

- 集成个体应：好而不同

个体与集成 - 简单分析

- 考虑二分类问题，假设基分类器的错误率为：

$$P(h_i(\mathbf{x}) \neq f(\mathbf{x})) = \epsilon$$

- 假设集成通过简单投票法结合 T 个分类器，若有超过半数的基分类器正确则分类就正确

$$H(\mathbf{x}) = \text{sign} \left(\sum_{i=1}^T h_i(\mathbf{x}) \right)$$

个体与集成 – 简单分析

- 假设基分类器的错误率相互独立，则由Hoeffding不等式可得集成的错误率为：

$$\begin{aligned} P(H(\mathbf{x}) \neq f(\mathbf{x})) &= \sum_{k=0}^{\lfloor T/2 \rfloor} \binom{T}{k} (1-\epsilon)^k \epsilon^{T-k} \\ &\leq \exp\left(-\frac{1}{2}T(1-2\epsilon)^2\right) \end{aligned}$$

- 上式显示，在一定条件下，随着集成分类器数目的增加，集成的错误率将指数级下降，最终趋向于0

个体与集成 – 简单分析

- 上面的分析有一个关键假设：**基学习器的误差相互独立**
 - 现实任务中，个体学习器是为解决同一个问题训练出来的，显然不可能互相独立
 - 事实上，个体学习器的“准确性”和“多样性”本身就存在冲突
- 如何产生“好而不同”的个体学习器是集成学习研究的核心
- 集成学习大致可分为两大类：
 - 个体学习器间存在强依赖关系，需串行生成的**序列化方法**；Boosting
 - 个体学习器间不存在强依赖关系，可同时生成的**并行化方法**：Bagging和随机森林 (Random Forest)

第8章 集成学习

8.1 个体与集成

➡ 8.2 Boosting

8.3 Bagging与随机森林

8.4 结合策略

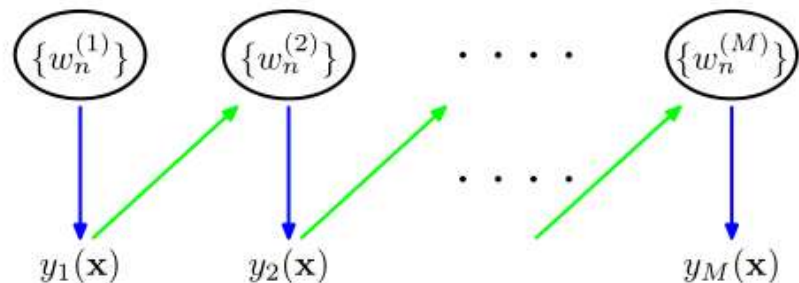
8.5 多样性

Boosting

□ 可将弱学习器提升为强学习器的算法

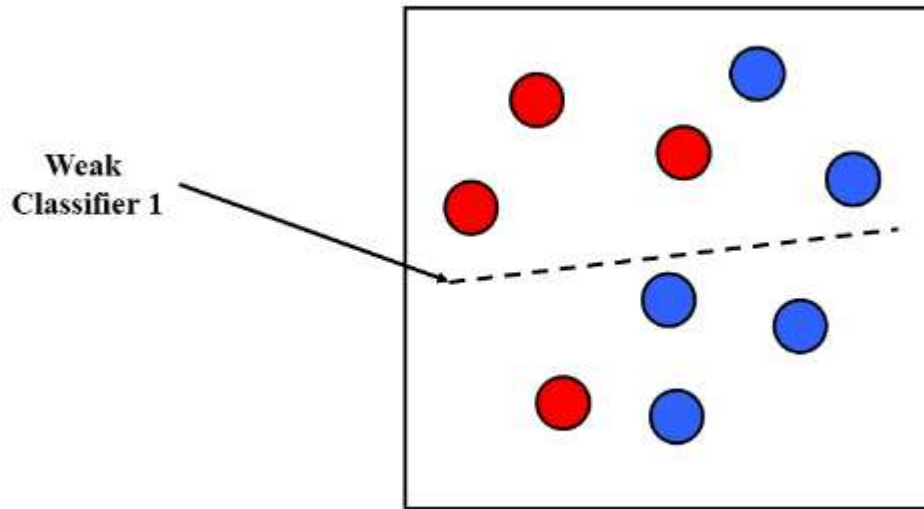
□ 工作机制：

- 先从初始训练集训练出一个基学习器
- 再根据基学习器的表现对训练样本进行调整，使得先前基学习器做错的训练样本在后续受到更多关注
- 然后再基于调整后的样本分布来训练下一个基学习器。
- 重复进行，直至基学习器数目达到事先指定的值T，最终将T个基学习器进行加权结合。



$$Y_M(\mathbf{x}) = \text{sign} \left(\sum_m^M \alpha_m y_m(\mathbf{x}) \right)$$

gif5.net



Boosting – AdaBoost算法

□ Boosting族算法最著名的代表是**AdaBoost** (Adaptive Boosting)

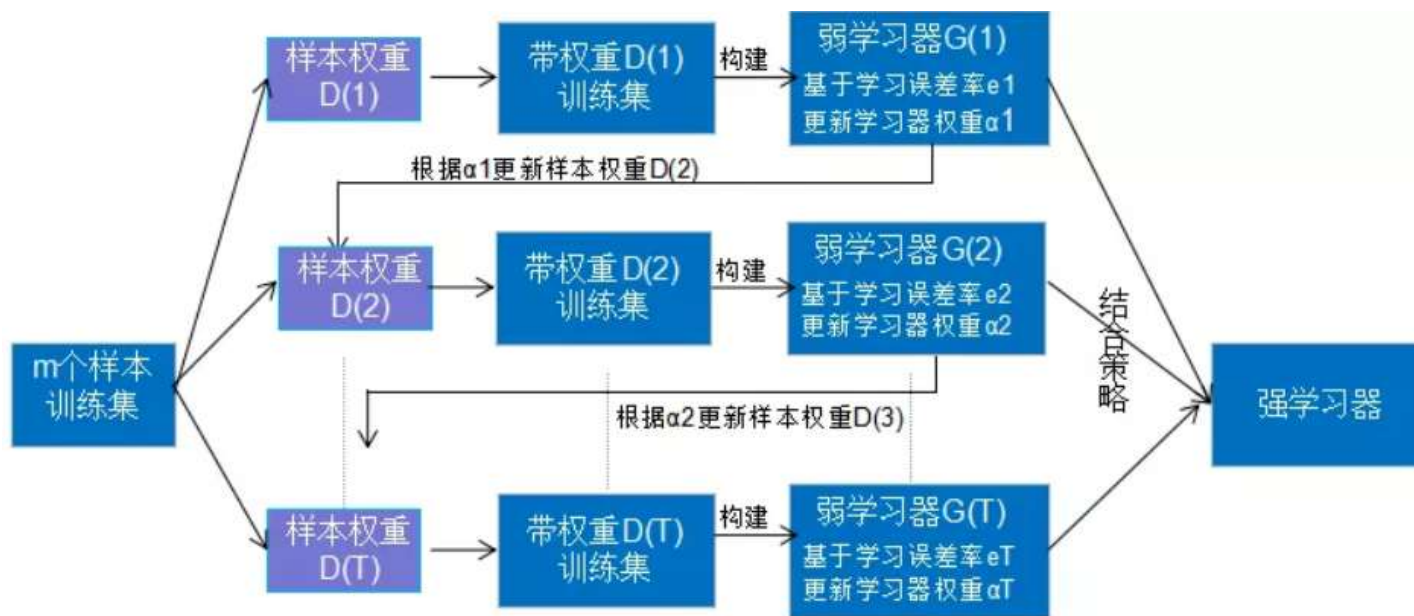
如上图，Boosting算法的思路：

(1) 样本权重表示样本分布，对特定的样本分布生成一个弱学习器。

(2) 根据该弱学习器模型的误差率 e 更新学习器权重 α 。

(3) 根据上一轮的学习器权重 α 来更新下一轮的样本权重。

(4) 重复步骤(1)(2)(3)，结合所有弱学习器模型，根据结合策略生成强学习器。



Boosting – AdaBoost算法

□ Boosting族算法最著名的代表是**AdaBoost** (Adaptive Boosting)

其中: $y_i \in \{-1, +1\}$, f 是真实函数。

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程:

- 1: $\mathcal{D}_1(\mathbf{x}) = 1/m$.
- 2: **for** $t = 1, 2, \dots, T$ **do**
- 3: $h_t = \mathcal{L}(D, \mathcal{D}_t)$;
- 4: $\epsilon_t = P_{\mathbf{x} \sim \mathcal{D}_t}(h_t(\mathbf{x}) \neq f(\mathbf{x}))$;
- 5: **if** $\epsilon_t > 0.5$ **then break**
- 6: $\alpha_t = \frac{1}{2} \ln \left(\frac{1-\epsilon_t}{\epsilon_t} \right)$;
- 7:
$$\mathcal{D}_{t+1}(\mathbf{x}) = \frac{\mathcal{D}_t(\mathbf{x})}{Z_t} \times \begin{cases} \exp(-\alpha_t), & \text{if } h_t(\mathbf{x}) = f(\mathbf{x}) \\ \exp(\alpha_t), & \text{if } h_t(\mathbf{x}) \neq f(\mathbf{x}) \end{cases}$$
$$= \frac{\mathcal{D}_t(\mathbf{x}) \exp(-\alpha_t f(\mathbf{x}) h_t(\mathbf{x}))}{Z_t}$$
- 8: **end for**

输出: $H(\mathbf{x}) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(\mathbf{x}) \right)$

初始化样本权值分布.
基于分布 $\mathcal{D}_t$ 从数据集 D 中训练出分类器 h_t .
估计 h_t 的误差.

确定分类器 h_t 的权重.

更新样本分布, 其中 Z_t 是规范化因子, 以确保 $\mathcal{D}_{t+1}$ 是一个分布.

训练数据的权重
弱学习器的权重

AdaBoost算法实例

- 例 给定如下表所示训练数据。假设个体学习器由 x （输入）和 y （输出）产生，其阈值 v （判定正反例的分界线）使该分类器在训练数据集上分类误差率最低。（ $y=1$ 为正例， $y=-1$ 为反例）

序号	1	2	3	4	5	6	7	8	9	10
x	0	1	2	3	4	5	6	7	8	9
y	1	1	1	-1	-1	-1	1	1	1	-1

- 第一个个体学习器：
我们首先认为 i （ $i=1,2,\dots,10$ ）的权重是一样的，即每一个数据同等重要。（权重是用来计算误差的）

x	0	1	2	3	4	5	6	7	8	9
w	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1	0.1
y	1	1	1	-1	-1	-1	1	1	1	-1

AdaBoost算法实例

(a) 在权值分布为 D_1 的训练数据上，阈值 v 取2.5（红线）时分类误差率最低（此时 $x=6,7,8$ 的数据被错分为反例，误差为它们的权重之和 $e_1 = 0.1 + 0.1 + 0.1 = 0.3$ ，误差率小于 $1/2$ 才有意义），故个体学习器为：

$$G_1(x) = \begin{cases} 1 & x < 2.5, \\ -1 & x > 2.5. \end{cases}$$

(b) 根据误差 e_1 计算系数 $\alpha_1 = 0.4236$ （公式： $\alpha_i = \frac{1}{2} \log \frac{1-e_i}{e_i}$ ），可以发现只有当 $e_1 < \frac{1}{2}$ 时， $\alpha_i > 0$ ，这样个体学习器才是有意义的）

(c) 更新训练数据的权值分布，公式如下：

$$w_{m+1,i} = \frac{w_{m,i}}{Z_m} \exp(-\alpha_m y_i G_m(x_i)), i = 1, 2, \dots, N,$$

$$Z_m = \sum_{i=1}^N w_{m,i} \exp(-\alpha_m y_i G_m(x_i)), Z_m \text{ 是为了保证每次权值总和为 } 1$$

（通过指数损失函数 $\exp(x)$ 调整权重，分类正确的降低权重 y_i 和 $G_m(x_i)$ 同号则 $y_i G_m(x_i) > 0, -\alpha_m y_i G_m(x_i) < 0$ ），分类错误的增加权重）

AdaBoost算法实例

```
>>> import math
>>> a = 0.5*math.log((1-0.3)/0.3, math.e)
>>> a
0.42364893019360184
>>> zm=0.1*math.e**(-0.4236)*7+0.1*math.e**(0.4236)*3
>>> zm
0.9165151400883117
>>> w1 = 0.1/zm*math.e**(-0.4236)
>>> w2 = 0.1/zm*math.e**(0.4236)
>>> w1,w2
(0.07143206644239734, 0.1666585116344062)
```

AdaBoost算法实例

D_2 (权重之和始终为1)

x	0	1	2	3	4	5	6	7	8	9
w	0.07143	0.07143	0.07143	0.07143	0.07143	0.07143	0.16667	0.16667	0.16667	0.07143
y	1	1	1	-1	-1	-1	1	1	1	-1

可以看到 $x=6,7,8$ 的数据的权重变大了，而其他数据的权重降低了，这是希望能把之前经常分类错误（经常分类错误会出现权重不断变大）的数据能在下一个个体学习器分类正确（记住：权重是用来计算误差的，为了降低误差，选择阈值时会倾向把权重大的分类正确）

$$f_1(x) = \alpha_1 G_1(x) = 0.4236 G_1(x)$$

$$0.4236 G_1(x) = \begin{cases} 0.4236 * 1 & x < 2.5, \\ 0.4236 * (-1) & x > 2.5. \end{cases}$$

$$\text{sign}[f_1(x)] = \begin{cases} 1 & x < 2.5, \\ -1 & x > 2.5. \end{cases}$$

集成学习器 $\text{sign}[f_1(x)]$ （第一次集成，只有一个个体学习器）在训练数据集上有3个误分类点

AdaBoost算法实例

□ 第二个个体学习器:

D_2

```
>>> a = 0.5*math.log((1-0.2143)/0.2143, math.e)
>>> a
0.6495990688511224
```

x	0	1	2	3	4	5	6	7	8	9
w	0.071 43	0.071 43	0.071 43	0.071 43	0.071 43	0.071 43	0.166 67	0.166 67	0.166 67	0.071 43
y	1	1	1	-1	-1	-1	1	1	1	-1

(a) 在权值分布为 D_2 的训练数据上, 阈值 v 取8.5时分类误差率最低 (此 $x=3,4,5$ 的数据被错分为正例, 误差为它们的权重之和 $e_2 = 0.07143 + 0.07143 + 0.07143 = 0.2143$, 误差率降低了!), 故个体学习器为:

$$G_2(x) = \begin{cases} 1 & x < 8.5, \\ -1 & x > 8.5. \end{cases}$$

(b) 根据误差 e_2 计算系数 $\alpha_2 = 0.6496$

(c) 更新训练数据的权值分布 (在 D_2 的基础上调整 D_3 , 分类正确的降低权重, 分类错误的增加权重):

AdaBoost算法实例

D_3

x	0	1	2	3	4	5	6	7	8	9
w	0.045 5	0.045 5	0.045 5	0.166 67	0.166 67	0.166 67	0.106 0	0.106 0	0.106 0	0.045 5
y	1	1	1	-1	-1	-1	1	1	1	-1

对比 D_2 可以看到 $x=3,4,5$ 的数据的权重变大了，而其他权重降低了。

$$0.4236G_1(x) = \begin{cases} 0.4236 * 1 & x < 2.5, \\ 0.4236 * (-1) & x > 2.5. \end{cases}$$

$$0.6496G_2(x) = \begin{cases} 0.6496 * 1 & x < 8.5, \\ 0.6496 * (-1) & x > 8.5. \end{cases}$$

$$f_2(x) = \alpha_1 G_1(x) + \alpha_2 G_2(x) = 0.4236G_1(x) + 0.6496G_2(x)$$

$$f_2(x) = \begin{cases} 0.4236 * 1 + 0.6496 * 1 = 1.0732 & x < 2.5, \\ 0.4236 * (-1) + 0.6496 * 1 = 0.226 & 2.5 < x < 8.5, \\ 0.6496 * (-1) = -0.6496 & x > 8.5, \end{cases}$$

$$\text{sign}[f_2(x)] = \begin{cases} 1 & x < 8.5, \\ -1 & x > 8.5. \end{cases}$$

AdaBoost算法实例

D_3

x	0	1	2	3	4	5	6	7	8	9
w	0.045 5	0.045 5	0.045 5	0.166 67	0.166 67	0.166 67	0.106 0	0.106 0	0.106 0	0.045 5
y	1	1	1	-1	-1	-1	1	1	1	-1

(a) 在权值分布为 D_3 的训练数据上，阈值 v 取5.5时分类误差率最低（ $e_3 = 0.1820$ ），误差率又降低了，故个体学习器为：

$$G_3(x) = \begin{cases} -1 & x < 5.5, \\ 1 & x > 5.5. \end{cases}$$

(b) 根据误差 e_3 计算系数 $\alpha_3 = 0.7514$

(c) 更新训练数据的权值分布

AdaBoost算法实例

D_4

x	0	1	2	3	4	5	6	7	8	9
w	0.125	0.125	0.125	0.102	0.102	0.102	0.065	0.065	0.065	0.125
y	1	1	1	-1	-1	-1	1	1	1	-1

$$f_3(x) = \alpha_1 G_1(x) + \alpha_2 G_2(x) + \alpha_3 G_3(x)$$

$$f_3(x) = \begin{cases} 0.3218 & x < 2.5, \\ -0.5254 & 2.5 < x < 5.5, \\ 0.9774 & 5.5 < x < 8.5, \\ -0.3218 & x > 8.5. \end{cases} \quad \text{sign}[f_3(x)] = \begin{cases} 1 & x < 2.5, \\ -1 & 2.5 < x < 5.5, \\ 1 & 5.5 < x < 8.5, \\ -1 & x > 8.5. \end{cases}$$

x	0	1	2	3	4	5	6	7	8	9
w	0.125	0.125	0.125	0.102	0.102	0.102	0.065	0.065	0.065	0.125
y	1	1	1	-1	-1	-1	1	1	1	-1

分类器 $\text{sign}[f_3(x)]$ 在训练数据集上有0个误分类点 (amazing!)

AdaBoost

- 对比三个个体学习器可以发现，权值很低的数据从侧面说明，它们在前面的学习器经常被分类正确，也就是说它们被分类正确的票数就比较多（ a 相当于每个分类器的票数），那么之后的个体学习器把它们分类错也影响不大，反正总票数是分类正确的票数多就可以了。
 - 例如 $x=1$ ，前面两次分类对了，获得正确票数 $0.4236+0.6496=1.0732$ ，第三次错了，获得错误票数 0.7514 ，正确票数多，最终还是分类正确了。
- 为了想办法让分类错误的数据变为分类正确的，后面的个体学习器也在努力。如 $x=6$ ，第一次分类错误的票数为 0.4236 ，第二次分类正确的票数 0.6496 ，可以看到为了让前面分类错误的数据变为分类正确的，后面个体学习器的重要性（ a ）需要比前面的大。

Boosting – AdaBoost推导

- 通过基学习器的线性组合

$$H(\mathbf{x}) = \sum_{t=1}^T \alpha_t h_t(\mathbf{x})$$

- 来最小化指数损失函数

$$\ell_{\text{exp}}(H \mid \mathcal{D}) = \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H(\mathbf{x})}]$$

Boosting – AdaBoost推导

- 若 $H(x)$ 能令指数损失函数最小化，则上式对 $H(x)$ 的偏导值为0，即

$$\frac{\partial \ell_{\text{exp}}(H \mid \mathcal{D})}{\partial H(\mathbf{x})} = -e^{-H(\mathbf{x})} P(f(\mathbf{x}) = 1 \mid \mathbf{x}) + e^{H(\mathbf{x})} P(f(\mathbf{x}) = -1 \mid \mathbf{x})$$

- 令上式得0可解得：

$$H(\mathbf{x}) = \frac{1}{2} \ln \frac{P(f(\mathbf{x}) = 1 \mid \mathbf{x})}{P(f(\mathbf{x}) = -1 \mid \mathbf{x})}$$

- 因此有：

$$\begin{aligned} \text{sign}(H(\mathbf{x})) &= \text{sign}\left(\frac{1}{2} \ln \frac{P(f(\mathbf{x}) = 1 \mid \mathbf{x})}{P(f(\mathbf{x}) = -1 \mid \mathbf{x})}\right) \\ &= \begin{cases} 1, & P(f(\mathbf{x}) = 1 \mid \mathbf{x}) > P(f(\mathbf{x}) = -1 \mid \mathbf{x}) \\ -1, & P(f(\mathbf{x}) = 1 \mid \mathbf{x}) < P(f(\mathbf{x}) = -1 \mid \mathbf{x}) \end{cases} \\ &= \arg \max_{y \in \{-1, 1\}} P(f(\mathbf{x}) = y \mid \mathbf{x}), \end{aligned}$$

这意味着 $\text{sign}(H(x))$ 达到了贝叶斯最优错误率，说明指数损失函数是分类任务原来0/1损失函数的一致的替代函数。

Boosting – AdaBoost推导

- 第一个基分类器 h_1 是通过直接将基学习算法用于初始数据分布而得；此后迭代地生成 h_t 和 α_t 。当基分类器 h_t 基于分布 D_t 产生后，该基分类器的权重 α_t 应使得 $\alpha_t h_t$ 最小化指数损失函数

$$\begin{aligned}\ell_{\text{exp}}(\alpha_t h_t \mid \mathcal{D}_t) &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_t} \left[e^{-f(\mathbf{x}) \alpha_t h_t(\mathbf{x})} \right] \\ &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_t} \left[e^{-\alpha_t} \mathbb{I}(f(\mathbf{x}) = h_t(\mathbf{x})) + e^{\alpha_t} \mathbb{I}(f(\mathbf{x}) \neq h_t(\mathbf{x})) \right] \\ &= e^{-\alpha_t} P_{\mathbf{x} \sim \mathcal{D}_t}(f(\mathbf{x}) = h_t(\mathbf{x})) + e^{\alpha_t} P_{\mathbf{x} \sim \mathcal{D}_t}(f(\mathbf{x}) \neq h_t(\mathbf{x})) \\ &= e^{-\alpha_t} (1 - \epsilon_t) + e^{\alpha_t} \epsilon_t, \quad \epsilon_t = P_{\mathbf{x} \sim \mathcal{D}_t}(h_t(\mathbf{x}) \neq f(\mathbf{x}))\end{aligned}$$

- 令指数损失函数的导数为0，即

$$\frac{\partial \ell_{\text{exp}}(\alpha_t h_t \mid \mathcal{D}_t)}{\partial \alpha_t} = -e^{-\alpha_t} (1 - \epsilon_t) + e^{\alpha_t} \epsilon_t$$

解得： $\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$

这恰是图8.3中算法第6行的分类器权重更新公式. 弱学习权重系数 α 。由上式可知，学习器误差率越小，则权重系数越大。

Boosting – AdaBoost推导

- 在获得 H_{t-1} 之后的样本分布进行调整, 使得下一轮的基学习器 h_t 能纠正 H_{t-1} 的一些错误, 理想的 h_t 能纠正 H_{t-1} 的全部错误, 即最小化:

$$\begin{aligned}\ell_{\text{exp}}(H_{t-1} + h_t \mid \mathcal{D}) &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})(H_{t-1}(\mathbf{x})+h_t(\mathbf{x}))}] \\ &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} e^{-f(\mathbf{x})h_t(\mathbf{x})}]\end{aligned}$$

- 可使用 $e^{-f(\mathbf{x})h_t(\mathbf{x})}$ 的泰勒展开近似为

$$\begin{aligned}\ell_{\text{exp}}(H_{t-1} + h_t \mid \mathcal{D}) &\simeq \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} \left(1 - f(\mathbf{x})h_t(\mathbf{x}) + \frac{f^2(\mathbf{x})h_t^2(\mathbf{x})}{2} \right) \right] \\ &= \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} \left(1 - f(\mathbf{x})h_t(\mathbf{x}) + \frac{1}{2} \right) \right]\end{aligned}$$

Boosting – AdaBoost推导

□ 于是，理想的基学习器：

$$\begin{aligned}h_t(\mathbf{x}) &= \arg \min_h \ell_{\text{exp}}(H_{t-1} + h \mid \mathcal{D}) \\&= \arg \min_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} \left(1 - f(\mathbf{x})h(\mathbf{x}) + \frac{1}{2} \right) \right] \\&= \arg \max_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} f(\mathbf{x})h(\mathbf{x}) \right] \\&= \arg \max_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\frac{e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}]} f(\mathbf{x})h(\mathbf{x}) \right],\end{aligned}$$

□ 注意到 $\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}]$ 是一个常数，令 $\mathbf{D}_t$ 表示一个分布：

$$\mathcal{D}_t(\mathbf{x}) = \frac{\mathcal{D}(\mathbf{x})e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}]}$$

Boosting – AdaBoost推导

□ 根据数学期望的定义，这等价于令：

$$\begin{aligned} h_t(\mathbf{x}) &= \arg \max_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\frac{e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}}[e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}]} f(\mathbf{x})h(\mathbf{x}) \right] \\ &= \arg \max_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_t} [f(\mathbf{x})h(\mathbf{x})] . \end{aligned}$$

□ 由 $f(x), h(x) \in \{-1, +1\}$ 有：

$$f(\mathbf{x})h(\mathbf{x}) = 1 - 2 \mathbb{I}(f(\mathbf{x}) \neq h(\mathbf{x}))$$

Boosting – AdaBoost推导

- 则理想的基学习器

$$h_t(\mathbf{x}) = \arg \min_h \mathbb{E}_{\mathbf{x} \sim \mathcal{D}_t} [\mathbb{I}(f(\mathbf{x}) \neq h(\mathbf{x}))]$$

- 由此可见，理想的 h_t 将在分布 $\mathcal{D}_t$ 下最小化分类误差. 因此，弱分类器将基于分布 $\mathcal{D}_t$ 来训练，且针对 $\mathcal{D}_t$ 的分类误差应小于0.5
- 这在一定程度上类似“残差逼近”的思想. 考虑到 $\mathcal{D}_t$ 和 $\mathcal{D}_{t+1}$ 的关系，有最终的**样本分布更新公式**

$$\begin{aligned} \mathcal{D}_{t+1}(\mathbf{x}) &= \frac{\mathcal{D}(\mathbf{x}) e^{-f(\mathbf{x})H_t(\mathbf{x})}}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [e^{-f(\mathbf{x})H_t(\mathbf{x})}]} \\ &= \frac{\mathcal{D}(\mathbf{x}) e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})} e^{-f(\mathbf{x})\alpha_t h_t(\mathbf{x})}}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [e^{-f(\mathbf{x})H_t(\mathbf{x})}]} \\ &= \mathcal{D}_t(\mathbf{x}) \cdot e^{-f(\mathbf{x})\alpha_t h_t(\mathbf{x})} \frac{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [e^{-f(\mathbf{x})H_{t-1}(\mathbf{x})}]}{\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} [e^{-f(\mathbf{x})H_t(\mathbf{x})}]} \end{aligned}$$

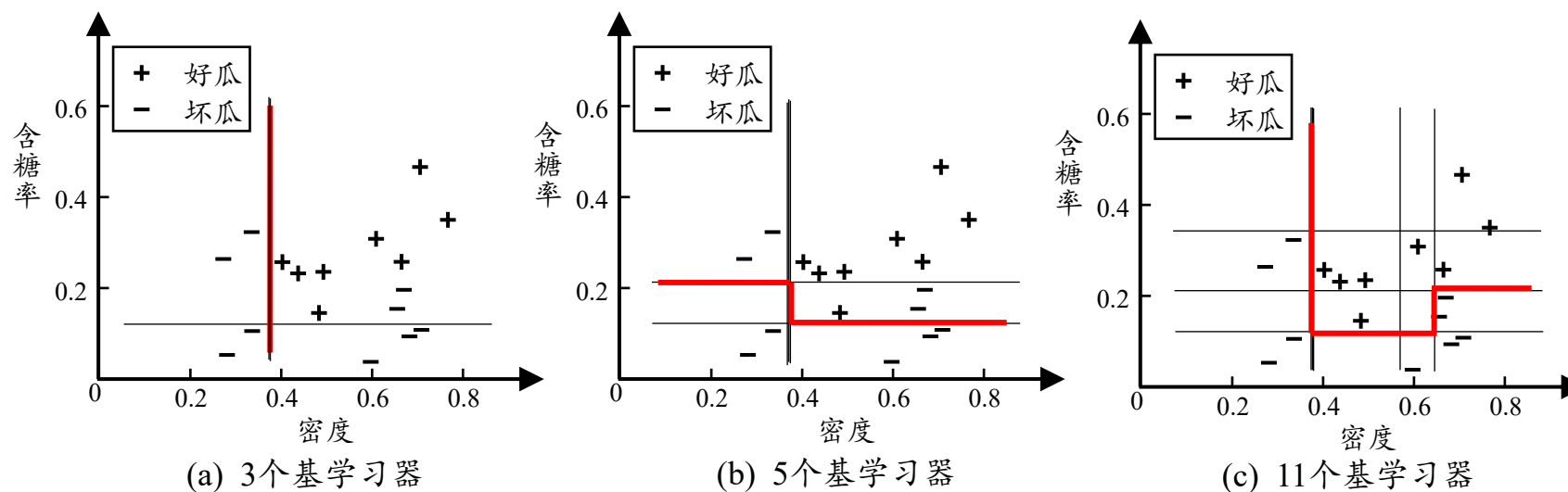
这恰是图8.3中算法**第7行**的**样本分布更新公式**.

Boosting – AdaBoost注意事项

- Boosting算法要求基学习器能对特定的数据分布进行学习，可通过**重赋权法**实施
- 对无法接受带权样本的基学习算法，通过**重采样法**处理
- 可以采用重启动，避免训练过程过早停止

Boosting – AdaBoost实验

- 从偏差-方差的角度：Boosting关注降低偏差，因此，可对泛化性能相当弱的学习器构造出很强的集成
- 以决策树桩为基学习器，在西瓜数据集3.0a上运行AdaBoost算法，不同规模的集成与基学习器对应的分类边界如下图：



西瓜数据集3.0a上 AdaBoost集成规模为3、5、11时，集成(红色)与基学习器(黑色)的分类边界。

第8章 集成学习

8.1 个体与集成

8.2 Boosting

➡ 8.3 Bagging与随机森林

8.4 结合策略

8.5 多样性

Bagging与随机森林

- 欲得到泛化性能强的集成，集成中的个体学习器应尽可能相互独立；虽然“独立”在现实任务中无法做到，但可以设法使基学习器尽可能具有较大的差异。
- 给定一个训练数据集，一种可能的做法是对训练样本进行采样，产生出若干个不同的子集，再从每个数据子集中训练出一个基学习器。这样，由于训练数据不同，我们获得的基学习器可望具有比较大的差异。
- 然而，为获得好的集成，我们同时还希望个体学习器不能太差。如果采样出的每个子集都完全不同，则每个基学习器只用到了一小部分训练数据，甚至不足以进行有效学习，这显然无法确保产生出比较好的基学习器。为解决这个问题，我们可考虑使用相互有交叠的采样子集。
- **Bagging**是并行式集成学习方法最著名的代表。

Bagging与随机森林

□ 自助采样法(bootstrap sampling)

- 给定包含 m 个样本的数据集，我们先随机取出一个样本放入采样集中，再把该样本放回初始数据集，使得下次采样时该样本仍有可能被选中，这样，经过 m 次随机采样操作，我们得到含 m 个样本的采样集，初始训练集中有的样本在采样集里多次出现，有的则从未出现。
- 由式(2.1)可知，初始训练集中约有63.2%的样本出现在采样集中。

□ Bagging算法

- 照这样，我们可采样出 T 个含 m 个训练样本的采样集，然后基于每个采样集训练出一个基学习器，再将这些基学习器进行结合。这就是Bagging的基本 流程。
- 在对预测输出进行结合时，Bagging通常对分类任务使用简单投票法，对即每个基学习器使用相回归任务使用简单平均法。

Bagging与随机森林 - Bagging算法

□ Bagging算法

- 给定包含 m 个样本的数据集，我们先随机取出一个样本放入采样集中，再把该样本放回初始数据集，使得下次采样时该样本仍有可能被选中

输入：训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
训练轮数 T .

过程：

1: **for** $t = 1, 2, \dots, T$ **do**
2: $h_t = \mathcal{L}(D, \mathcal{D}_{bs})$
3: **end for**

输出： $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y)$

$\mathcal{D}_{bs}$ 是自助采样产生的
样本分布.

Bagging 算法

Bagging与随机森林 - Bagging算法特点

□ 时间复杂度低

- 假定基学习器的计算复杂度为 $O(m)$ ，考虑到采样与投票/平均过程的复杂度为 $O(s)$ ，则bagging的复杂度大致为 $T(O(m)+O(s))$
- 由于 $O(s)$ 很小且 T 是一个不大的常数
- 因此训练一个bagging集成与直接使用基学习器的复杂度同阶。说明Bagging是一个很高效的集成学习算法

- ## □ 自助采样过程还给Bagging带来了来一个优点：
- 由于每个基学习器只使用了初始训练集中约63.2%的样本，剩余约36.8%的样本可用作验证集来对泛化性能进行“包外估计”。为此需记录每个基学习器所使用的训练样本。

Bagging与随机森林 - 包外估计

- 不妨令 D_t 表示 h_t 实际使用的训练样本集，令 $H^{oob}(x)$ 表示对样本 x 的包外预测，即仅考虑那些未使用样本 x 训练的基学习器在 x 上的预测

$$H^{oob}(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\mathbf{x}) = y) \cdot \mathbb{I}(\mathbf{x} \notin D_t)$$

- Bagging泛化误差的包外估计为：

$$\epsilon^{oob} = \frac{1}{|D|} \sum_{(\mathbf{x}, y) \in D} \mathbb{I}(H^{oob}(\mathbf{x}) \neq y)$$

Bagging与随机森林- Bagging实验

- 事实上,包外样本还有许多其他用途.例如当基学习器是决策树时,可使用 包外样本来辅助剪枝,或用于估计决策树中各结点的后验概率以辅助对零训练 样本结点的处理;当基学习器是神经网络时,可使用包外样本来辅助早期停止 以减小过拟合风险.
- 从偏差-方差的角度: **Bagging**主要关注降低方差,在不剪枝的决策树、神经网络等易受样本影响的学习器上效果更好。
- 我们以基于信息增益划分的决策树为基学习器,在表4 .5的西瓜数据集3 .0 a上运行 **Bagging**算法,不同规模的集成及其基学习器所对应的分类边界如图8.6所示.

Bagging与随机森林- Bagging实验

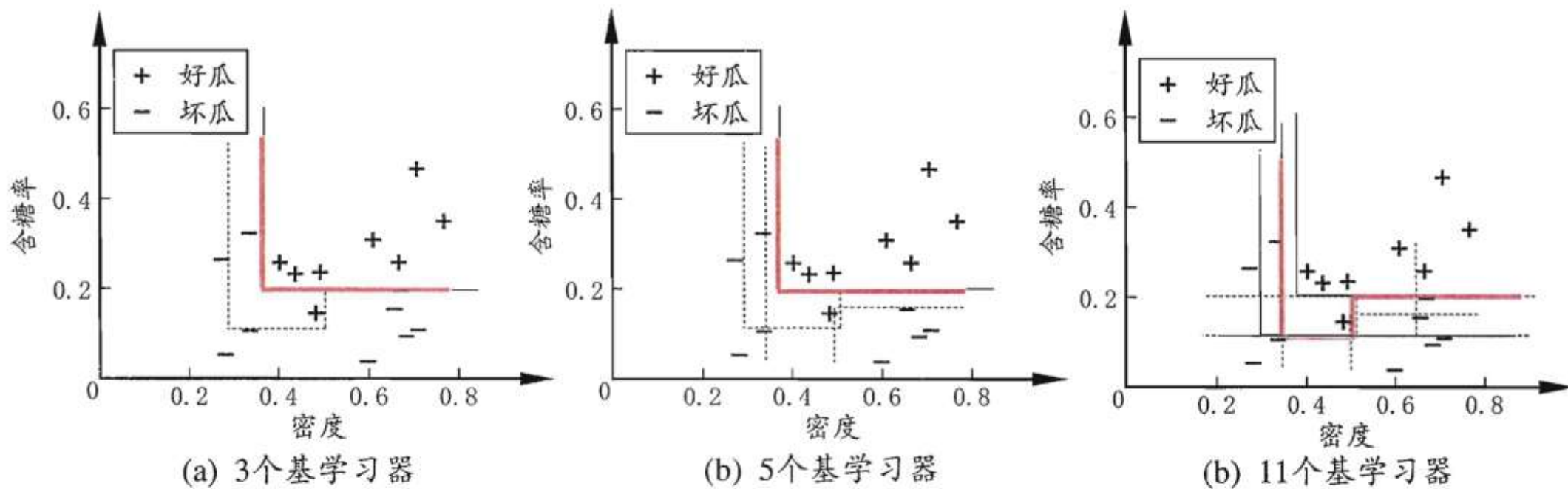


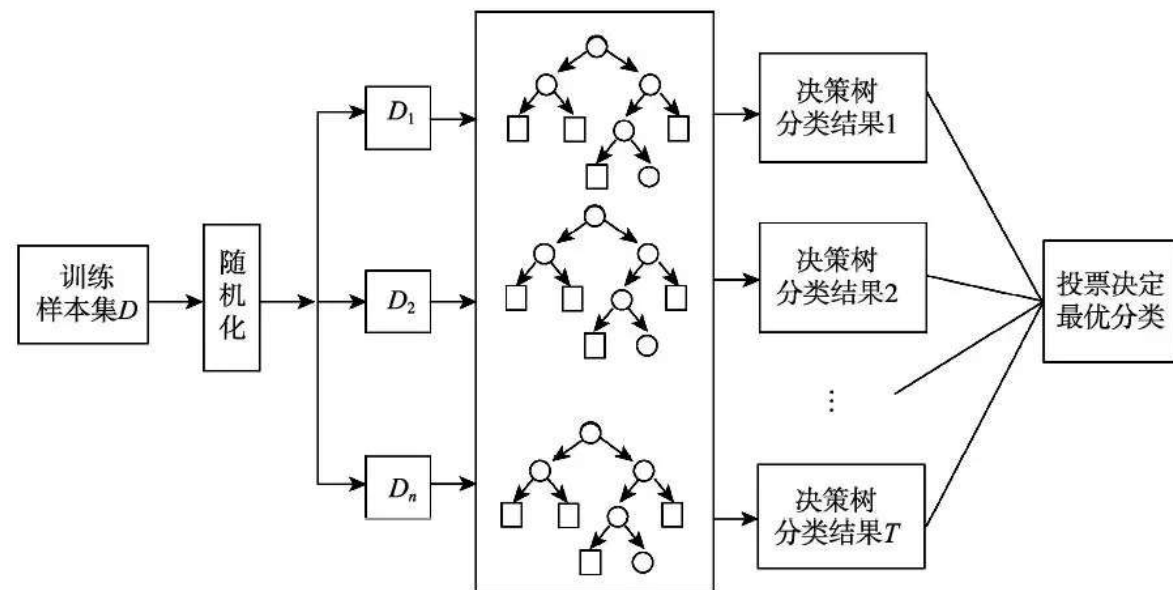
图 8.6 西瓜数据集 3.0 α 上 Bagging 集成规模为 3、5、11 时, 集成(红色)与基学习器(黑色)的分类边界.

Bagging与随机森林-随机森林

- 随机森林(Random Forest, 简称RF)是bagging的一个扩展变种
- 在以决策树为基学习器构建bagging集成的基础上, 进一步在决策树的训练过程中引入随机属性选择。
 - ✓ 采样的随机性
 - ✓ 属性选择的随机性

Bagging与随机森林-随机森林

- 决策树的一个主要缺点在于经常对训练数据过拟合。随机森林是解决这个问题的一种方法。
- 随机森林本质上是许多决策树的集合，其中每棵树都和其他树略有不同。随机森林背后的思想是，每棵树的预测可能都相对较好，但可能对部分数据过拟合。如果构造很多树，并且每棵树的预测都很好，但都以不同的方式过拟合，那么我们可以对这些树的结果取平均值来降低过拟合。既能减少过拟合又能保持树的预测能力，这可以在数学上严格证明。
- 为了实现这一策略，我们需要构造许多决策树。每棵树都应该对目标值做出可以接受的预测，还应该与其他树不同。随机森林的名字来自于将随机性添加到树的构造过程中，以确保每棵树都各不相同。
- 随机森林中树的随机化方法有两种：一种是通过选择用于构造树的数据点，另一种是通过选择每次划分测试的特征。



Bagging与随机森林 - 随机森林算法

□ 随机森林算法

Input: Data set $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
Feature subset size K .

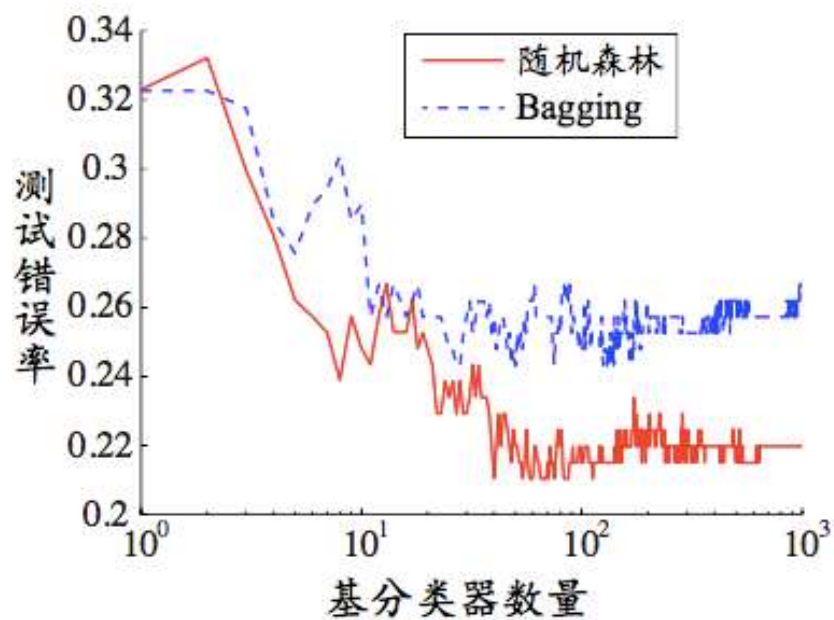
Process:

1. $N \leftarrow$ create a tree node based on D ;
2. **if** *all instances in the same class* **then return** N
3. $\mathcal{F} \leftarrow$ the set of features that can be split further;
4. **if** $\mathcal{F}$ *is empty* **then return** N
5. $\tilde{\mathcal{F}} \leftarrow$ select K features from $\mathcal{F}$ randomly;
6. $N.f \leftarrow$ the feature which has the best split point in $\tilde{\mathcal{F}}$;
7. $N.p \leftarrow$ the best split point on $N.f$;
8. $D_l \leftarrow$ subset of D with values on $N.f$ smaller than $N.p$;
9. $D_r \leftarrow$ subset of D with values on $N.f$ no smaller than $N.p$;
10. $N_l \leftarrow$ call the process with parameters (D_l, K) ;
11. $N_r \leftarrow$ call the process with parameters (D_r, K) ;
12. **return** N

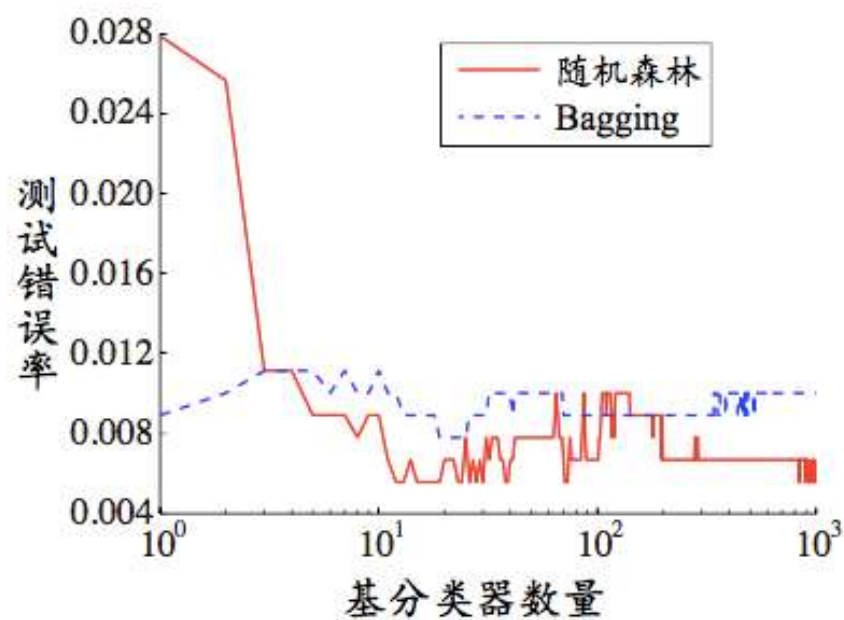
Output: A random decision tree

Bagging与随机森林 - 随机森林实验

- 随机森林算法简单、容易实现、计算开销小，它是很多现实任务中展现出强大的性能。
- 随机森林收敛性与Bagging相似。



(a) glass 数据集



(b) auto-mpg 数据集

第8章 集成学习

8.1 个体与集成

8.2 Boosting

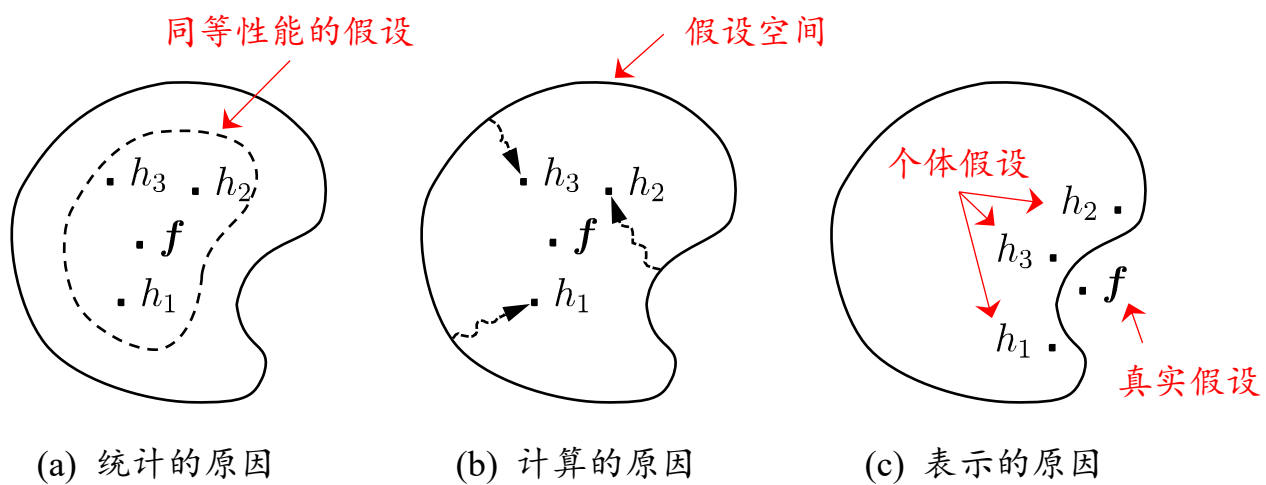
8.3 Bagging与随机森林

➡ 8.4 结合策略

8.5 多样性

结合策略

□ 学习器的组合可以从三个方面带来好处：



结合策略 - 平均法(用于回归)

□ 简单平均法

$$H(\mathbf{x}) = \frac{1}{T} \sum_{i=1}^T h_i(\mathbf{x}).$$

□ 加权平均法

$$H(\mathbf{x}) = \sum_{i=1}^T w_i h_i(\mathbf{x}), \quad w_i \geq 0 \text{ and } \sum_{i=1}^T w_i = 1.$$

结合策略 - 平均法

- 简单平均法是加权平均法的特例
- 加权平均法在二十世纪五十年代被广泛使用
- 集成学习中的各种结合方法都可以看成是加权平均法的变种或特例
- 加权平均法可认为是集成学习研究的基本出发点
- 加权平均法未必一定优于简单平均法

结合策略 - 投票法(用于分类)

- 绝对多数投票法 (majority voting)

$$H(\mathbf{x}) = \begin{cases} c_j & \text{if } \sum_{i=1}^T h_i^j(\mathbf{x}) > \frac{1}{2} \sum_{k=1}^l \sum_{i=1}^T h_i^k(\mathbf{x}) \\ \text{rejection} & \text{otherwise.} \end{cases}$$

- 相对多数投票法 (plurality voting)

$$H(\mathbf{x}) = c_{\arg \max_j \sum_{i=1}^T h_i^j(\mathbf{x})}$$

- 加权投票法 (weighted voting)

$$H(\mathbf{x}) = c_{\arg \max_j \sum_{i=1}^T w_i h_i^j(\mathbf{x})}$$

结合策略 - 学习法

□ Stacking是学习法的典型代表

Input: Data set $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
First-level learning algorithms $\mathcal{L}_1, \dots, \mathcal{L}_T$;
Second-level learning algorithm $\mathcal{L}$.

Process:

```
1. for  $t = 1, \dots, T$ :    % Train a first-level learner by applying the
2.    $h_t = \mathcal{L}_t(D)$ ;    % first-level learning algorithm  $\mathcal{L}_t$ 
3. end
4.  $D' = \emptyset$ ;          % Generate a new data set
5. for  $i = 1, \dots, m$ :
6.   for  $t = 1, \dots, T$ :
7.     $z_{it} = h_t(\mathbf{x}_i)$ ;
8.   end
9.    $D' = D' \cup ((z_{i1}, \dots, z_{iT}), y_i)$ ;
10. end
11.  $h' = \mathcal{L}(D')$ ;      % Train the second-level learner  $h'$  by applying
                           % the second-level learning algorithm  $\mathcal{L}$  to the
                           % new data set  $D'$ .
```

Output: $H(\mathbf{x}) = h'(h_1(\mathbf{x}), \dots, h_T(\mathbf{x}))$

多响应线性回归(MLR)作为次级学习器的学习算法 效果较好

□ 贝叶斯模型平均(BMA)

第8章 集成学习

8.1 个体与集成

8.2 Boosting

8.3 Bagging与随机森林

8.4 结合策略

➡ 8.5 多样性

多样性 - 误差-分歧分解

- 定义学习器 h_i 的分歧(ambiguity):

$$A(h_i \mid \mathbf{x}) = (h_i(\mathbf{x}) - H(\mathbf{x}))^2$$

- 集成的分歧定义为:

$$\begin{aligned}\bar{A}(h \mid \mathbf{x}) &= \sum_{i=1}^T w_i A(h_i \mid \mathbf{x}) \\ &= \sum_{i=1}^T w_i (h_i(\mathbf{x}) - H(\mathbf{x}))^2\end{aligned}$$

多样性 - 误差-分歧分解

- 分歧项代表了个体学习器在样本 x 上的不一致性，即在一定程度上反映了个体学习器的多样性，个体学习器 h_i 和集成 H 的平方误差分别为

$$E(h_i | \mathbf{x}) = (f(\mathbf{x}) - h_i(\mathbf{x}))^2$$

$$E(H | \mathbf{x}) = (f(\mathbf{x}) - H(\mathbf{x}))^2$$

多样性 - 误差-分歧分解

□ 令 $\bar{E}(h | \mathbf{x}) = \sum_{i=1}^T w_i \cdot E(h_i | \mathbf{x})$ 表示个体学习器误差的加权均值，有

$$\begin{aligned}\bar{A}(h | \mathbf{x}) &= \sum_{i=1}^T w_i E(h_i | \mathbf{x}) - E(H | \mathbf{x}) \\ &= \bar{E}(h | \mathbf{x}) - E(H | \mathbf{x}) .\end{aligned}$$

□ 上式对所有样本 $\mathbf{x}$ 均成立，令 $p(\mathbf{x})$ 表示样本的概率密度，则在全样本上有

$$\sum_{i=1}^T w_i \int \bar{A}(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x} = \sum_{i=1}^T w_i \int E(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x} - \int E(H | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

多样性 - 误差-分歧分解

- 个体学习器 h_i 在全样本上的泛化误差和分歧项分别为：

$$E_i = \int E(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

$$A_i = \int A(h_i | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

- 集成的泛化误差为：

$$E = \int E(H | \mathbf{x}) p(\mathbf{x}) d\mathbf{x}$$

- 令 $\overline{E} = \sum_{i=1}^T w_i E_i$ 表示个体学习器泛化误差的加权均值，
 $\overline{A} = \sum_{i=1}^T w_i A_i$ 表示个体学习器的加权分歧值，有

$$E = \overline{E} - \overline{A}$$

多样性 - 误差-分歧分解

- 这个漂亮的式子显示: 个体学习器精确性越高、多样性越大, 则集成效果越好。称为误差-分歧分解
- 为什么不能直接把 $\bar{E} - \bar{A}$ 作为优化目标来求解?
 - 现实任务中很难直接对 $\bar{E} - \bar{A}$ 进行优化,
 - 它们定义在整个样本空间上
 - $\bar{A}$ 不是一个可直接操作的多样性度量
 - 上面的推导过程只适用于回归学习, 难以直接推广到分类学习任务上去

多样性 - 多样性度量

- 多样性度量(diversity measure)用于度量集成中个体学习器的多样性
- 对于二分类问题，分类器 h_i 与 h_j 的预测结果联立表(contingency table)为

	$h_i = +1$	$h_i = -1$
$h_j = +1$	a	c
$h_j = -1$	b	d

$$a + b + c + d = m$$

多样性 - 多样性度量

□ 常见的多样性度量

- 不合度量(Disagreement Measure)

$$dis_{ij} = \frac{b + c}{m}$$

- 相关系数(Correlation Coefficient)

$$\rho_{ij} = \frac{ad - bc}{\sqrt{(a + b)(a + c)(c + d)(b + d)}}$$

多样性 - 多样性度量

□ 常见的多样性度量

- Q-统计量 (Q-Statistic)

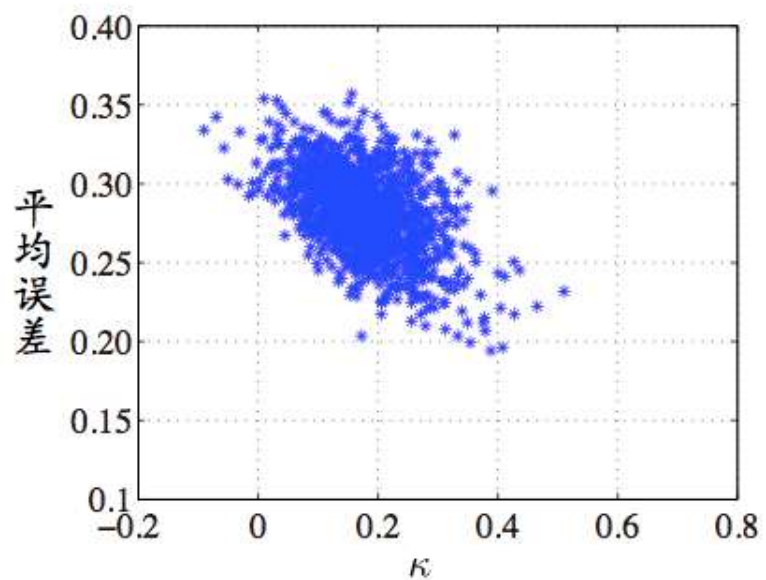
$$Q_{ij} = \frac{ad - bc}{ad + bc} \qquad |Q_{ij}| \leq |\rho_{ij}|$$

- K-统计量 (Kappa-Statistic)

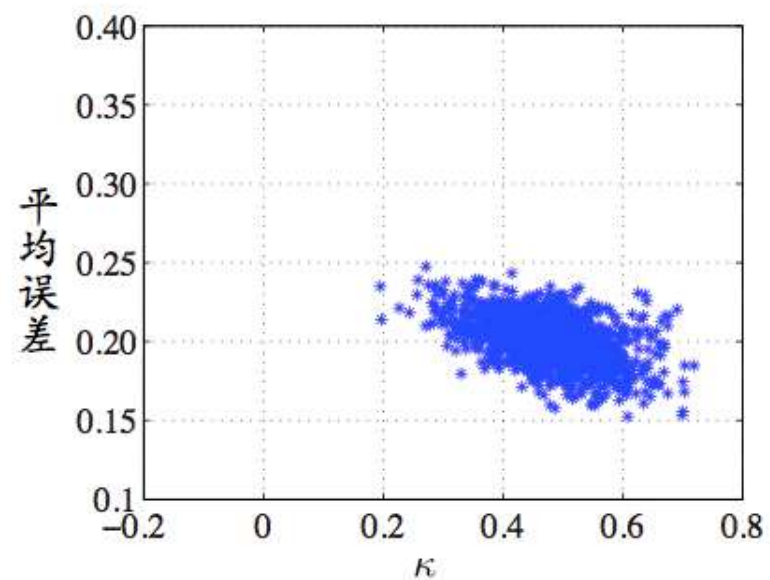
$$\kappa = \frac{p_1 - p_2}{1 - p_2} \quad \rightarrow \quad \begin{aligned} p_1 &= \frac{a + d}{m}, \\ p_2 &= \frac{(a + b)(a + c) + (c + d)(b + d)}{m^2} \end{aligned}$$

多样性 - 多样性度量

□ k -误差图



(a) AdaBoost 集成



(b) Bagging 集成

多样性 - 多样性增强

□ 常见的增强个体学习器的多样性的方法

- 数据样本扰动
- 输入属性扰动
- 输出表示扰动
- 算法参数扰动

多样性 - 多样性增强 - 数据样本扰动

□ 数据样本扰动通常是基于采样法

- Bagging中的自助采样法
- Adaboost中的序列采样

数据样本扰动对“不稳定基学习器”很有效

□ 对数据样本的扰动敏感的基学习器(不稳定基学习器)

- 决策树，神经网络等

□ 对数据样本的扰动不敏感的基学习器(稳定基学习器)

- 线性学习器，支持向量机，朴素贝叶斯，k近邻等

多样性 - 多样性增强 - 输入属性扰动

□ 随机子空间算法(random subspace)

输入: 训练集 $D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_m, y_m)\}$;
基学习算法 $\mathcal{L}$;
基学习器数 T ;
子空间属性数 d' .

过程:

- 1: **for** $t = 1, 2, \dots, T$ **do**
- 2: $\mathcal{F}_t = \text{RS}(D, d')$
- 3: $D_t = \text{Map}_{\mathcal{F}_t}(D)$
- 4: $h_t = \mathcal{L}(D_t)$
- 5: **end for**

输出: $H(\mathbf{x}) = \arg \max_{y \in \mathcal{Y}} \sum_{t=1}^T \mathbb{I}(h_t(\text{Map}_{\mathcal{F}_t}(\mathbf{x})) = y)$

多样性 - 多样性增强 - 输出表示扰动

- 翻转法(Flipping Output)
- 输出调剂法(Output Smearing)
- ECOC法

多样性 - 多样性增强 - 算法参数扰动

- 负相关法
- 不同的多样性增强机制同时使用

随机森林

- 在scikit-learn中, RandomForest的分类类是RandomForestClassifier, 回归是RandomForestRegressor, 需要调参的参数包括两部分, 第一部分是Bagging框架的参数, 第二部分是CART决策树的参数。
- 最主要的两个参数是n_estimators和max_features。
- `class sklearn.ensemble.RandomForestClassifier(n_estimators=10, criterion='gini', max_depth=None, min_samples_split=2, min_samples_leaf=1, min_weight_fraction_leaf=0.0, max_features='auto', max_leaf_nodes=None, min_impurity_split=1e-07, bootstrap=True, oob_score=False, n_jobs=1, random_state=None, verbose=0, warm_start=False, class_weight=None)`

RandomForest参数

- ❑ `n_estimators`: 表示森林里树的个数。理论上是越大越好。但是伴随着就是计算时间的增长。但是并不是取得越大就会越好，预测效果最好的将会出现在合理的树个数。
- ❑ `max_features`: 随机选择特征集合的子集合，并用来分割节点。子集合的个数越少，方差就会减少的越快，但同时偏差就会增加的越快。根据较好的实践经验，如果是回归问题则：`max_features=n_features`，如果是分类问题则`max_features=sqrt(n_features)`。如果想获取较好的结果，必须将`max_depth=None`, 同时`min_sample_split=1`。同时还要进行`cross_validated`（交叉验证），除此之外在random forest中，`bootstrap=True`。


```
from sklearn.datasets import make_blobs
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.tree import DecisionTreeClassifier

X, y = make_blobs(n_samples=10000, n_features=10, centers=100,
...               random_state=0)
clf = DecisionTreeClassifier(max_depth=None, min_samples_split=2,
...                          random_state=0) #定义一个决策树分类器
scores = cross_val_score(clf, X, y)
scores.mean() # 0.97... #这里是决策树的模型精准度得分
clf = RandomForestClassifier(n_estimators=10, max_depth=None,
...                          min_samples_split=2, random_state=0) #定义一个随机森林分类器
scores = cross_val_score(clf, X, y)
scores.mean() # 0.999... 随机森林训练器的模型精确度得分
clf = ExtraTreesClassifier(n_estimators=10, max_depth=None,
...                        min_samples_split=2, random_state=0) #定义一个极端森林分类器
scores = cross_val_score(clf, X, y)
scores.mean() > 0.999 #True #极端森林训练器的模型精确度得分, 效果优于随机森林
```

Stacking利器-mlxtend库

```
1  from sklearn import datasets
2
3  iris = datasets.load_iris()
4  X, y = iris.data[:, 1:3], iris.target
5
6  from sklearn import model_selection
7  from sklearn.linear_model import LogisticRegression
8  from sklearn.neighbors import KNeighborsClassifier
9  from sklearn.naive_bayes import GaussianNB
10 from sklearn.ensemble import RandomForestClassifier
11 from mlxtend.classifier import StackingClassifier
12 import numpy as np
13
14 clf1 = KNeighborsClassifier(n_neighbors=1)
15 clf2 = RandomForestClassifier(random_state=1)
16 clf3 = GaussianNB()
17 lr = LogisticRegression()
```

Stacking利器-mlxtend库

```
sclf = StackingClassifier(classifiers=[clf1, clf2, clf3],
                          meta_classifier=lr)

print('3-fold cross validation:\n')

for clf, label in zip([clf1, clf2, clf3, sclf],
                      ['KNN',
                       'Random Forest',
                       'Naive Bayes',
                       'StackingClassifier']):

    scores = model_selection.cross_val_score(clf, X, y,
                                              cv=3, scoring='accuracy')
    print("Accuracy: %0.2f (+/- %0.2f) [%s]"
          % (scores.mean(), scores.std(), label))
```

Xgboost

- ❑ xgboost的全称是eXtreme Gradient Boosting。是 Gradient Boosting Machine的一个c++实现，作者为机器学习大牛陈天奇。



- ❑ xgboost最大的特点在于，它能够自动利用CPU的多线程进行并行，同时在算法上加以改进提高了精度。
- ❑ 作者在研究中深感自己受制于现有库的计算速度和精度，因此开始着手搭建xgboost项目，并逐渐成型。它的处女秀是Kaggle的希格斯子信号识别竞赛，因为出众的效率与较高的预测准确度在比赛论坛中引起了参赛选手的广泛关注。

Xgboost

- ❑ 下载地址: <https://www.lfd.uci.edu/~gohlke/pythonlibs/#xgboost>
- ❑ 安装: `pip install xgboost-0.80-cp37-cp37m-win_amd64.whl`

Xgboost, a distributed gradient boosting (GBDT, GBRT or GBM) library.

Requires the Microsoft Visual C++ Redistributable for Visual Studio 2017

[xgboost-0.80-cp27-cp27m-win32.whl](#)

[xgboost-0.80-cp27-cp27m-win_amd64.whl](#)

[xgboost-0.80-cp34-cp34m-win32.whl](#)

[xgboost-0.80-cp34-cp34m-win_amd64.whl](#)

[xgboost-0.80-cp35-cp35m-win32.whl](#)

[xgboost-0.80-cp35-cp35m-win_amd64.whl](#)

[xgboost-0.80-cp36-cp36m-win32.whl](#)

[xgboost-0.80-cp36-cp36m-win_amd64.whl](#)

[xgboost-0.80-cp37-cp37m-win32.whl](#)

[xgboost-0.80-cp37-cp37m-win_amd64.whl](#)

Xgboost代码

```
import xgboost

# First XGBoost model for Pima Indians dataset
from numpy import loadtxt
from xgboost import XGBClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

# load data
dataset = loadtxt('pima-indians-diabetes.csv', delimiter=",")
# split data into X and y
X = dataset[:,0:8]
Y = dataset[:,8]
# split data into train and test sets
seed = 7
test_size = 0.33
X_train, X_test, y_train, y_test = train_test_split(X, Y, test_size=test_size,
random_state=seed)
# fit model no training data
model = XGBClassifier()
model.fit(X_train, y_train)
# make predictions for test data
y_pred = model.predict(X_test)
predictions = [round(value) for value in y_pred]
# evaluate predictions
accuracy = accuracy_score(y_test, predictions)
print("Accuracy: %.2f%%" % (accuracy * 100.0))
```

GB、GBDT、Xgboost

- GBDT和xgboost在竞赛和工业界使用都非常频繁。
- GB、GBDT、xgboost之间有非常紧密的联系，GBDT是以决策树（CART）为基学习器的GB算法，xgboost扩展和改进了GBDT，xgboost算法更快，准确率也相对高一些。

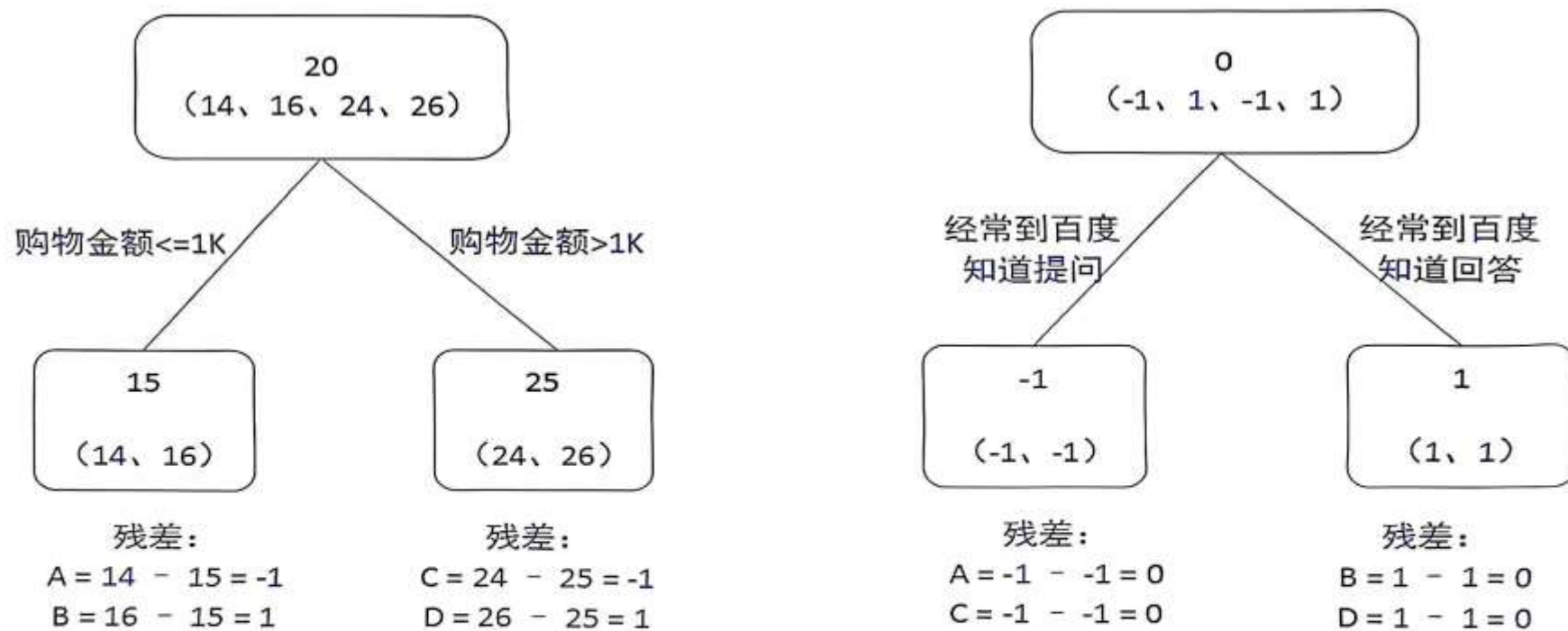
- 机器学习中的学习算法的目标是为了优化或者说最小化损失函数
- Gradient boosting的思想是迭代生多个（M个）弱的模型，然后将每个弱模型的预测结果相加，后面的模型 $F_{m+1}(x)$ 基于前面学习模型的 $F_m(x)$ 的效果生成的，关系如下：

$$1 \leq m \leq M \quad F_{m+1}(x) = F_m(x) + h(x)$$

GBDT

- GB算法中最典型的基学习器是决策树，尤其是**CART**，正如名字的含义，**GBDT**是**GB**和**DT**的结合。
- 要注意的是这里的决策树是回归树，**GBDT**中的决策树是个弱模型，深度较小一般不会超过**5**，叶子节点的数量也不会超过**10**，对于生成的每棵决策树乘上比较小的缩减系数（学习率 <0.1 ），有些**GBDT**的实现加入了随机抽样（`subsample 0.5 $\leq$ f $\leq$ 0.8`）提高模型的泛化能力。通过交叉验证的方法选择最优的参数。
- **GBDT**中使用的是回归树，分枝时衡量最好的标准不再是**GINI**系数，而是最小化均方差。
- **Xgboost**是GB算法的高效实现。

GBDT



以残差 (-1, 1, -1, 1) 为样本拟合一棵回归树

A的预测值 = 树1左节点 值 15 + 树2左节点 -1 = 14
B的预测值 = 树1左节点 值 15 + 树2右节点 1 = 16
依次可得C和D预测值

GBDT-例子

如下表所示：一组数据，特征为年龄、体重，身高为标签值。共有5条数据，前四条为训练样本，最后一条为要预测的样本。

编号	年龄(岁)	体重 (kg)	身高(m)(标签值)
0	5	20	1.1
1	7	30	1.3
2	21	70	1.7
3	30	60	1.8
4(要预测的)	25	65	?

GBDT-例子

□ 1.初始化弱学习器

$$f_0(x) = \operatorname{argmin}_c \sum_{i=1}^N L(y_i, c)$$

- 损失函数为平方损失，因为平方损失函数是一个凸函数，直接求导，导数等于零，得到c。

$$\sum_{i=1}^N \frac{\partial L(y_i, c)}{\partial c} = \sum_{i=1}^N \frac{\partial (\frac{1}{2} (y_i - c)^2)}{\partial c} = \sum_{i=1}^N c - y_i$$

令导数为零

$$\sum_{i=1}^N c - y_i = 0 \rightarrow c = (\sum_{i=1}^N y_i) / N$$

- 所以初始化时，c取值为所有训练样本标签的均值。
 $c = (1.1 + 1.3 + 1.7 + 1.8) / 4 = 1.475$ ，此时得到初始学习器 $f_0(x)$

$$f_0(x) = c = 1.475$$

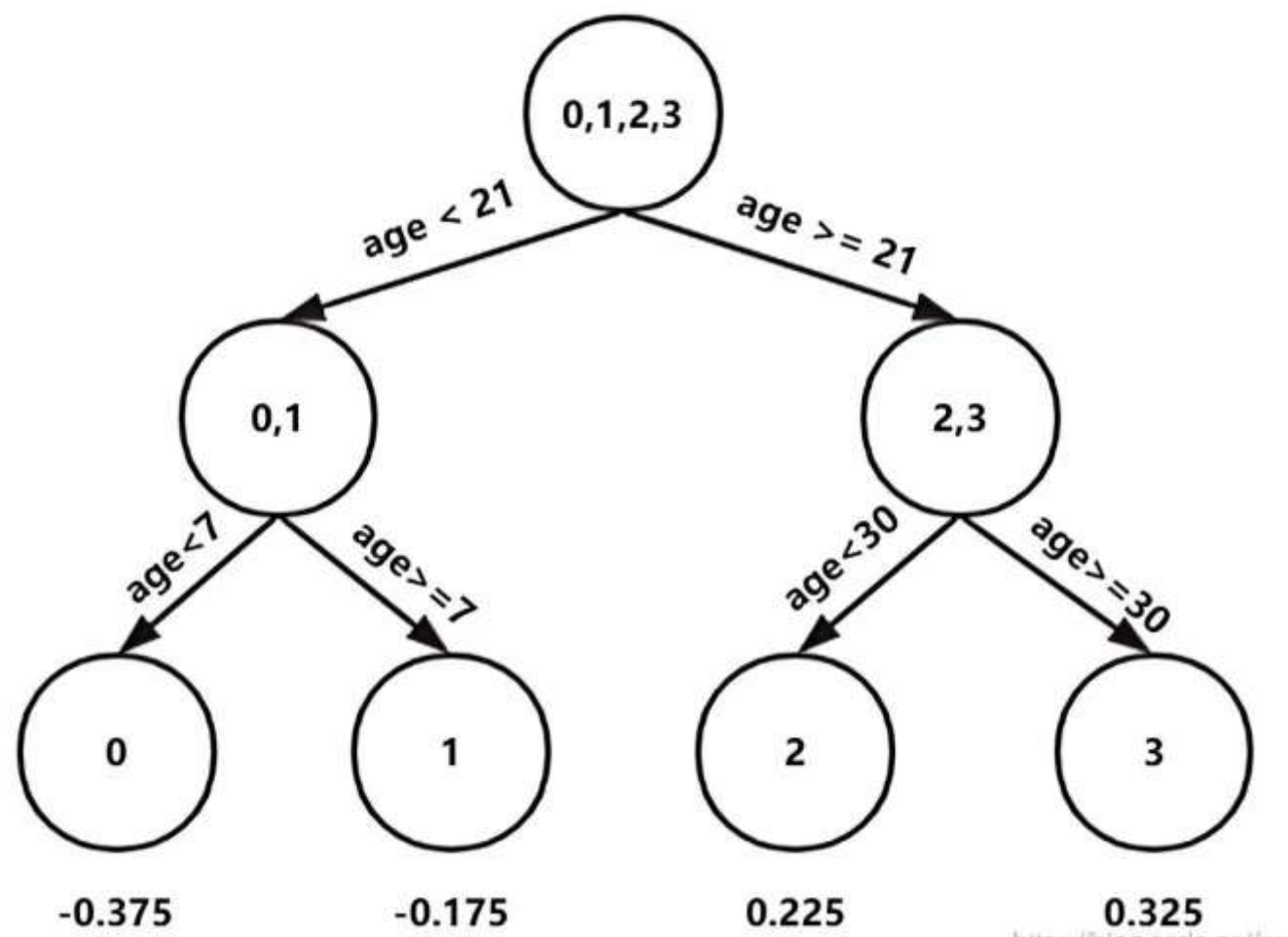
GBDT-例子

□ 残差在下表列出：

编号	真实值	$f_0(x)$	残差
0	1.1	1.475	-0.375
1	1.3	1.475	-0.175
2	1.7	1.475	0.225
3	1.8	1.475	0.325

□ 此时将残差作为样本的真实值来训练弱学习器 $f_1(x)$ ，即下表数据

编号	年龄(岁)	体重(kg)	标签值
0	5	20	-0.375
1	7	30	-0.175
2	21	70	0.225
3	30	60	0.325



<https://blog.csdn.net/zn>

GBDT-例子

- 得到最后的强学习器：

$$f(x) = f_5(x) = f_0(x) + \sum_{m=1}^5 \sum_{j=1}^4 \gamma_{jm} I(x \in R_{jm})$$

- 预测样本5

$$f_0(x) = 1.475$$

在 $f_1(x)$ 中，样本4的年龄为25，大于划分节点21但小于30，所以被预测为0.2250

在 $f_2(x)$ 中，样本4的...此处省略...所以被预测为0.2025

为什么是0.2025？这是根据第二颗树得到的，可以GitHub简单运行一下代码

在 $f_3(x)$ 中，样本4的...此处省略..所以被预测为0.1823

在 $f_4(x)$ 中，样本4的.此处省略...所以被预测为0.1640

在 $f_5(x)$ 中，样本4的..此处省略...所以被预测为0.1476

最终预测结果：

$$f(x) = 1.475 + 0.1 * (0.225 + 0.2025 + 0.1823 + 0.164 + 0.1476) = 1.56714$$

GBDT VS Xgboost

- ❑ GBDT以CART作为基分类器，xgboost还支持线性分类器，这个时候xgboost相当于带L1和L2正则化项的逻辑回归（分类问题）或者线性回归（回归问题）。
- ❑ 传统GBDT在优化时只用到一阶导数信息，xgboost则对代价函数进行了二阶泰勒展开，同时用到了一阶和二阶导数。xgboost工具支持自定义代价函数，只要函数可一阶和二阶求导。
- ❑ xgboost在代价函数里加入了正则项，用于控制模型的复杂度。正则项里包含了树的叶子节点个数、每个叶子节点上输出的score的L2模的平方和。从Bias-variance tradeoff角度来讲，正则项降低了模型的variance，使学习出来的模型更加简单，防止过拟合，这也是xgboost优于传统GBDT的一个特性。

GBDT VS Xgboost

- ❑ **Shrinkage**（缩减），相当于学习速率（xgboost中的eta）。xgboost在进行完一次迭代后，会将叶子节点的权重乘上该系数，主要是为了削弱每棵树的影响，让后面有更大的学习空间。实际应用中，一般把eta设置得小一点，然后迭代次数设置得大一点。
- ❑ **列抽样（column subsampling）**。xgboost借鉴了随机森林的做法，支持列抽样，不仅能降低过拟合，还能减少计算，这也是xgboost异于传统gbdt的一个特性。
- ❑ **对缺失值的处理**。对于特征的值有缺失的样本，xgboost可以自动学习出它的分裂方向。

GBDT VS Xgboost

- ❑ Xgboost工具支持并行。
 - Xgboost的并行不是tree粒度的并行，Xgboost也是一次迭代完才能进行下一次迭代的（第t次迭代的代价函数里包含了前面t-1次迭代的预测值）。xgboost的并行是在特征粒度上的。
- ❑ 决策树的学习最耗时的一个步骤就是对特征的值进行排序（因为要确定最佳分割点）
- ❑ Xgboost在训练之前，预先对数据进行了排序，然后保存为block结构，后面的迭代中重复地使用这个结构，大大减小计算量。
 - 这个block结构也使得并行成为了可能，在进行节点的分裂时，需要计算每个特征的增益，最终选增益最大的那个特征去做分裂，那么各个特征的增益计算就可以开多线程进行。

阅读材料

- 集成学习方面的主要推荐读物是[Zhou,2012]，本章提及的所有内容在该书中都有更深入的详细介绍。[Kuncheva,2004;Rockach,2010b]可供参考，[Schapire and Freund,2012]则是专门关于Boosting的著作，集成学习方面有一些专门性的会议MCS(International Workshop on Multiple Classifier System)。
- Boosting源于[Schapire,1990]对[Kearns and Valiant,1989]提出的“弱分类器是否等价于强学习”这个重要理论问题的构造性证明。最初的Boosting算法仅有理论意义，经数年努力后[Freund and Schapire,1997]提出Adaboost，并因此或得理论计算机科学方面的重要奖项—哥德尔奖。关于Boosting和Bagging已有的很多理论研究成果参阅[Zhou,2012]第2-3章。



重庆大学
CHONGQING UNIVERSITY

THANKS
祝学习快乐!